



MAKERERE UNIVERSITY

**COLLEGE OF COMPUTING AND INFORMATION SCIENCES
SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY**

**A REPORT ON FIELD ATTACHMENT/ INTERNSHIP AT
NEPTUNE FINANCIAL SOFTWARE GROUP**

Field Attachment Period (16th June – 08th August 2025)

BY

KARAMAGI REAGAN

11/U/16268/EVE

Field attachment Report submitted to the school of computing and Informatics Technology

In Partial fulfilment of the requirements for the DEGREE BACHELOR OF SCIENCE IN
SOFTWARE ENGINEERING of Makerere University Kampala.

Name & Signature of Student

Name & Sign of Academic Supervisor

(Date)

Name & Sign of Field Supervisor

(Stamp & Date)

¹ KARAMAGI REAGAN

DECLARATION PAGE

I, KARAMAGI REAGAN, hereby declare that the presented report of internship is uniquely prepared by me after working as an intern for a period of EIGHT (8) WEEKS from 16th June to 8th August 2025 under Neptune FINANCIAL SOFTWARE.

I, also confirm the authenticity/ originality of the report, with indication that acknowledgement was accordingly done in form of citations, quotations and references to other people's ideas/work where used.

I, also confirm that the report has been prepared only for my academic requirement and not for any other purpose.

Date:

ACKNOWLEDGEMENT PAGE

Any attempt at any level cannot be satisfactorily completed without the support and guidance of learned people. My Internship Report is a result of something more than hard work and dedication. It is a true outcome of encouragement, support and guidance and constructive criticism of different people that I acknowledge with immense pleasure. I would like to express my gratitude to all those who made the completion of this report a reality. I dedicate my gratitude to Makerere University for giving me permission to commence this report work, to do the necessary research work. I gratefully acknowledge my deepest sense of gratitude to my revered and intellectual supervisor... (SAMSON SERWANDA) who provided me with all necessary suggestions and guiding me throughout my internship period.

Additionally, I highly acknowledge the greatest altitude, positive open-minded colleagues of the department of technicians and also extend to the other departments of the Neptune Software Group. I thank the great pleasure of a friendly, family likely nature and high resilience attained in that every member seems to care of every one in their presence, provide help or assistance where necessarily and their effort to see I practice and learn during the internship through their organizational culture to understand the organizational workflow.

I thank my parents for always being on my side throughout this time period and providing me with all kind of emotional and financial support.

ABSTRACT

It is not possible to estimate the practical situations while studying in a classroom, an internship is a platform where one can get huge stuff to learn and the best chance to handle the professional situation. People are more than happy to give you work and teach you how it is done. If you wait for work to be given to you, you'll end up sitting idle and learn nothing. The problem-solving concept is not what we learn from our books, it is much beyond that. Internships are an opportunity to apply the knowledge of the problem solving practically. During the course of my internship. One must learn during the internship how to work with the team to bring productive results to the organization and how to Co-ordinate with the team and maintain communication with other departments for smooth flow of work. You will be influenced by a good work culture and to operate as a team. College life is quite different world from professional life. Here documents the Java backend programming, Application Programming Interface (API), Rest APIs, software tools software frameworks like DataGrip, IntelliJ for mini project and program designing and implementation. Work environment really matters a lot. If you have cooperative and helpful people around you, it gets easier to get the job done. Most of the work was given to me by Field Supervisor. He used to make sure that I always have some or the other work to do and was always happy to teach us something new. In conclusion the internship program has been very enlightening and I have managed to learn a lot and to put into practice what I have studied. The report concludes with the internship mini project on Change Request Form System in Appendices.

TABLE OF CONTENTS

Table of Contents

DECLARATION PAGE	2
ACKNOWLEDGEMENT PAGE	3
ABSTRACT	4
TABLE OF CONTENTS	5
List of Figures	7
List of Tables.....	8
List of acronyms/ Abbreviations	9
1.0. CHAPTER ONE: INTRODUCTION	10
1.1. Introduction	10
1.3. Objectives of the Field Attachment	10
1.4. Background of the Organization of Field Attachment	11
1.5. Organization Culture	13
1.6. The structure of the organization	13
1.7. The main activities of the organization and ongoing IT projects	14
2. CHAPTER TWO: STUDENT’S EXPERIENCES	15
2.1. Title or Position occupied in an organisations	15
2.2. Duties and responsibilities	15
2.3. Supervision levels and relationship with Supervisor	16
2.5. Working Relationship among team members/ other staffs	17
3.0. CHAPTER THREE: EVALUATION ON FIELD ATTACHMENT	18
3.0.1 Introduction to backend Programming	18
3.0.2. Structured Query Language	19
3.0.3. Java Language	20
3.0.4. Spring Boot Framework	20
3.0.5. Postman	21
3.0.6. DataGrip	22
3.0.7. IntelliJ	24
3.0.8. Restful API	25
3.8. Preparedness of the Agency to receive and manage Students for Field Attachment	31
3.9. Career Motivation	31
4.0. CHAPTER FOUR: CONCLUSIONS AND RECOMMENDATIONS	32
4.1. Conclusions	32

4.2. Recommendations	33
5.0. CHAPTER FIVE: REFERENCES	34
6.0. CHAPTER SIX: APPEDICES.....	36

List of Figures

FIGURE 1	13
FIGURE 2	14
FIGURE 3	16
FIGURE 4	19
FIGURE 5	19
FIGURE 6	19
FIGURE 7	25
FIGURE 8	28
FIGURE 9	28
FIGURE 10	36
FIGURE 11	37
FIGURE 12	38
FIGURE 13	39
FIGURE 14	40
FIGURE 15	40
FIGURE 16	41
FIGURE 17	41
FIGURE 18	42
FIGURE 19	42
FIGURE 20	43
FIGURE 21	44
FIGURE 22	44
FIGURE 23	45
FIGURE 24	45
FIGURE 25	46
FIGURE 26	46
FIGURE 27	47
FIGURE 28	47
FIGURE 29	48
FIGURE 30	48

List of Tables

TABLE 1	12
TABLE 2	15
TABLE 3	27
TABLE 4	29
TABLE 5	29
TABLE 6	29

List of acronyms/ Abbreviations

AI	Applicaton Interface
AOP	Aspect Oriented Programming
API	Application Programming Interface
APIs	Application Programming Interfaces
CRUD.....	CREATE READ UPDATE AND DELETE
DCL	Data Control Language
DDL	Data Definition Language
DI	Dependency Injection
DML.....	Data Manipulation Language
EE	Enterprise Edition
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Interface
IoC	Inversion of Control
JPA	Java Persistance Application
JVM	Java Virtual Machine
NoSQL	Not Only Structured Query Language
PostgreSQL	PostreStructured Qurey Language
RDBMS	Relational Database Management System
RDSMS	Relational Data Stream Management System
REST	Represtational State Transfer
Restful API	Representational State Transferful Application Programming Interface
SOAP.....	Software Oriented Application Programming
SQL	Structured Query Language
XML	Extended MarkUp Language

1.0. CHAPTER ONE: INTRODUCTION

1.1. Introduction

The field report is written document by KARAMAGI REAGAN of registration number 11/U/16268/EVE as student of MAKERERE UNIVERSITY. The student conducted the field Attachment known as Internship at Neptune Software Group for the period from 16th June to 8th August 2025. The report comprises the coverage concepts under taken at Neptune Software Group mainly the activities in learning the Java backend programming, embracing new software tools and frameworks.

1.2. Background of the Field Attachment

The field Attachment is field based practical work carried out for purposes of learning and researching outside the University control. Exposure to various activities, duties and responsibilities in field work organization to enhance the study programs to experience the real life of work.

The field Attachment constituted the procedural work progress of internship at Neptune Software Group where intensive positive change gradually happened. The change was due to the procedural concepts that I had attained during the College time and basics knowledge required revamped the challenge in that technology advancement needed. Therefore, I had to advance with the aid of the field Supervisor to learn new technology concepts that are on demand though basic skills were still required to build on.

With the daily interaction of the Field Supervisor provided daily learning progress to combat with latest skill technology in that basic building was tackled and made sure I grasp and understand. Through the interaction emphasized open minded skill because I looked nervous in most cases whenever he questioned about the related program concepts.

1.3. Objectives of the Field Attachment

Internships are generally thought of to be reserved for college students looking to gain experience in a particular field. However, a wide array of people can benefit from Training Internships in order to receive real world experience and develop their skills.

- To get hands on experience real life situation.
- To provide an opportunity to apply the principles and techniques theoretically learnt into real-life problem-solving situations.
- To provide an opportunity to interact with various stakeholders and potential employers.
- To appreciate field situations that generate information for curricula review and improvement.
- To develop understanding of work ethics, employment demands, responsibilities and opportunities.
- To enhance and strengthen linkages between the University and various stakeholders.
- Utilizing internships is a great way to build resume and develop skills that can be emphasized in resume for future jobs. When applying for a Training Internship, make sure to highlight any special skills or talents that can make you stand apart from the rest of the applicants so that you have an improved chance of landing the position.

1.4. Background of the Organization of Field Attachment

Neptune, founded in 1999 in London, has a rich history of delivering core banking systems to the banking and financial service sector. Having delivered end to end solutions to over 100 financial service institutions over the years, have the proven experience needed to empower our clients with high performance and scalable solutions that are deployed quickly and reliably. Have been working in this industry for the last 25 years.

Neptune focus is to harness in depth business domain knowledge and technical expertise to provide innovative end to end financial services solutions. Aim to entrust financial institutions with a sustainable competitive advantage by making them more resilient and effective. Thereby, be one step ahead of their competitors.

Neptune now operates local implementation cum marketing offices across East Africa (Kenya-Regional Head office, Tanzania and Uganda) and also an office in west Africa (Nigeria). With work experience, empower clients with quick yet reliable high performance and scalable solutions.

Neptune established its Indian operation at Chennai in 2006. At Chennai Centre, take care of product engineering global product support and RAD towards, the Neptune's flagship products of Rubikon & Orbit-R.

KARAMAGI REAGAN
FINAL INTERNSHIP 2025

Vision	Mission	Goals and Objectives	Values
Empowering clients with sustainable competitive advantage by providing innovative business software solutions and excellent services within a cost active bottom line.	• Delivering client success through innovation. • Harness our deep business domain knowledge and technical expertise to provide end to end integration of technology with business processes. • Bring trust and personal responsibility to all relationships.	• To deliver core banking systems to the financial services sector. • To harness in depth domain knowledge and technical expertise. • To provide innovative end to end financial services solution. • To entrust financial institutions with a sustainable competitive advantage	• Passion • Innovation • Excellence

Table 1

1.5. Organization Culture

Organizational culture refers to the shared values, beliefs, norms, behaviours, and practices that characterize a particular workplace or company. The culture is the collective personality of an organization that shapes the way employees interact, make decisions, and approach their work. Essentially, corporate culture serves as a blueprint for how business is done within the organization and influences the overall work environment and employee experiences.

Culture reflects the company's history, leadership style, and values and belonging among employees, as they collectively align their actions with the established norms and expectations. Alternatively, culture can be implicit, existing in the unspoken behaviors and attitudes of employees. Either way, system greatly influences employee perceptions of roles, camaraderie, collaboration, problem-solving approaches, and the organization's external reputation.

The organization's values are: Passion, Innovation and Excellence.

Neptune has a commendable work culture mainly focused on a growth mindset. The company is popular for employees dedicating their skills, time, and money to help make a difference. This difference contributes to employee diversity and inclusion, considering Neptune has employees from all backgrounds and walks of life working to ensure customer satisfaction. The company's work culture deeply aligns with its core values, which include of delivering core banking systems to the banking and financial services sector.

1.6. The structure of the organization

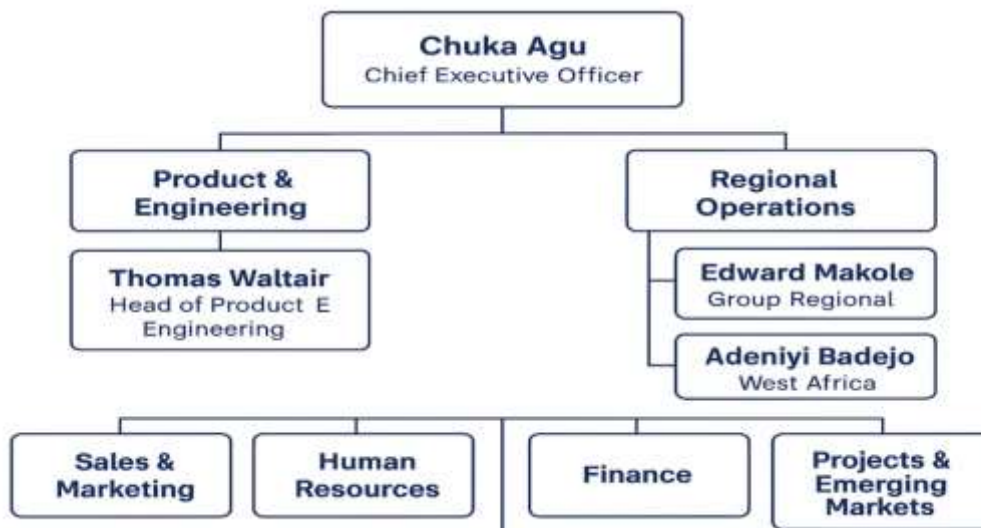


Figure 1

1.7. The main activities of the organization and ongoing IT projects

The main activities are financial software solutions automate business processes for banks and fintech companies by providing with a secure, regulatory compliant and scalable platforms to run their business efficiently. Neptune Software Group is a specialized provider of financial software solutions, primarily serving banks and fintech companies. Here's a breakdown of their **main activities**:

- ✓ **Banking Solutions:** They offer platforms tailored for retail banking, microfinance, treasury management, and digital banking.
- ✓ **Fintech Automation:** Their software automates business processes while ensuring scalability and regulatory compliance.

Neptune Software Group is actively evolving its platform through several **ongoing IT projects** in 2025, with a strong focus on AI, SAP modernization, and developer experience. Here's a snapshot of their current initiatives:

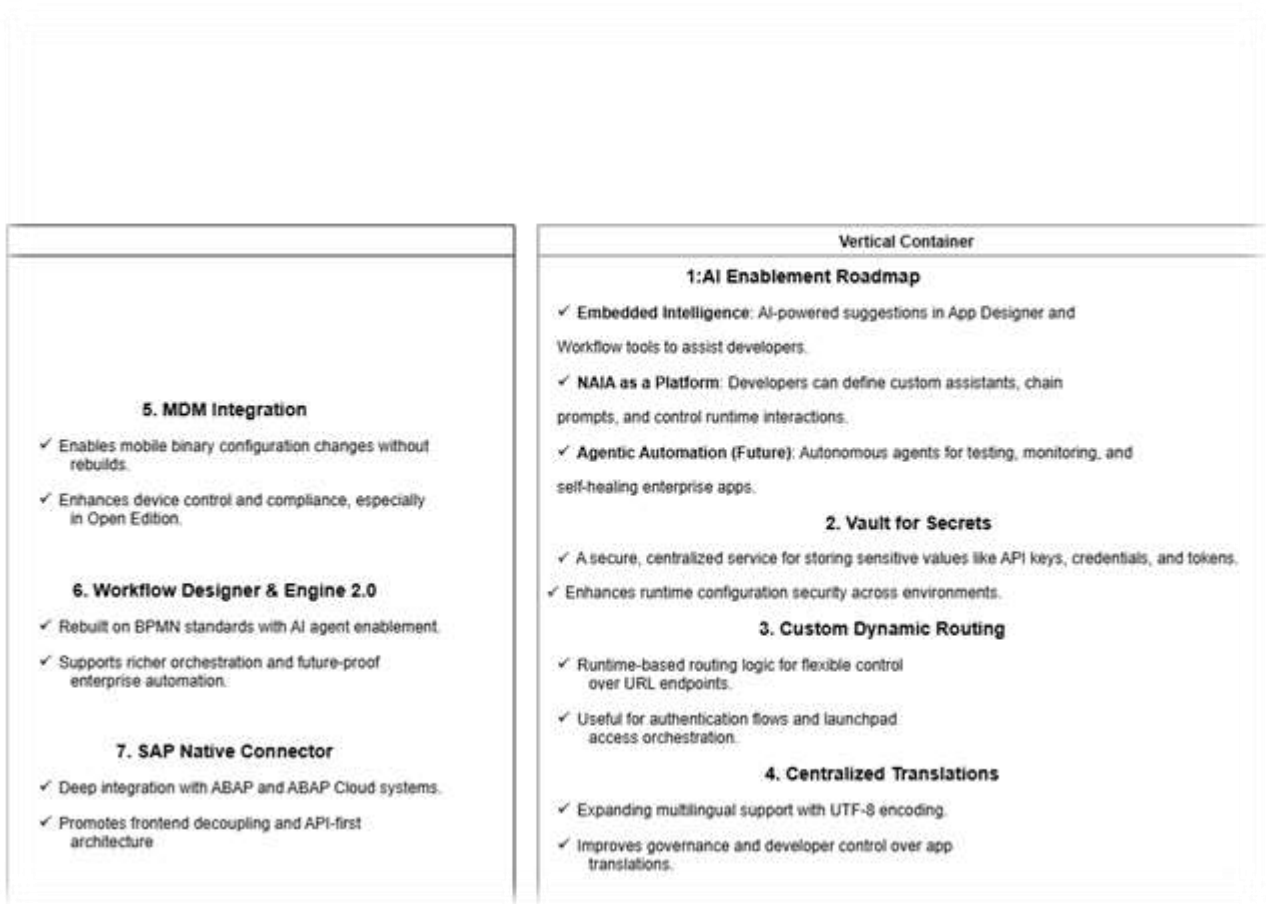


Figure 2

2. CHAPTER TWO: STUDENT'S EXPERIENCES

2.1. Title or Position occupied in an organisations

- ✓ Security.
- ✓ Accounts.
- ✓ Sales.
- ✓ Catering.
- ✓ Marketing.
- ✓ Head technician.
- ✓ Technician.

2.2. Duties and responsibilities

Duties and responsibilities are often used interchangeably, but they have distinct meanings. Duties refer to the specific tasks or actions that an individual is expected to perform as part of their job or role. These tasks are usually outlined in a job description and are directly related to the position held. On the other hand, responsibilities encompass a broader scope and refer to the obligations and expectations that come with a particular role. Responsibilities may include duties, but they also include the overall accountability, decision-making, and management of resources associated with the role. While duties are more task-oriented, responsibilities are more encompassing and reflect the overall expectations placed on an individual.

Duties refer to the specific tasks and actions that an individual is expected to perform as part of their job or role. These tasks are often outlined in job descriptions or work contracts and are directly related to the position held. Duties are typically more concrete and tangible, focusing on the execution of specific actions or activities. They are often assigned to individuals based on their skills, expertise, and qualifications.

Responsibilities, on the other hand, encompass a broader scope and refer to the obligations and expectations that individuals have towards their roles, colleagues, and the organization as a whole. Unlike duties, responsibilities are not always explicitly defined and can vary based on the context and organizational culture. They require individuals to be accountable for their actions and the impact they have on others and the organization. Responsibilities are often more abstract and require individuals to exercise judgment and discretion in their approach.

Duties	Responsibilities
Writing code, debugging programs, and collaborating with the team to develop software solutions.	Decision making, problem-solving, and taking ownership of outcomes.

Table 2

2.3. Supervision levels and relationship with Supervisor

Level of supervision is determined by the way in which work is assigned, it is reviewed, how it is reviewed, and what guidelines, prototypes and protocols are as below.

- Close Supervision — indicates that the internee is assigned duties according to specific procedures. Work is checked frequently, and in addition there may be formal training.
- Supervision — indicates that the internee performs a variety of routine duties within established policies and procedures or by referral to the supervisor's guidelines.
- General Supervision — indicates that the internee develops procedures for performance of variety of duties; or performs complex duties within established policy guidelines.
- Direction — indicates that the internee establishes procedures for attaining specific goals and objectives in a broad area of work. Only the final results of work done are typically reviewed. Incumbent typically develops procedures within the limits of established policy guidelines.
- General Direction — indicates that the internee receives guidance in terms of broad goals and overall objectives and is responsible for establishing the methods to attain them. Generally, the internee is in charge of an area of work, and typically formulates policy for this area but does not necessarily have final authority for approving policy.

2.4. Work Team and its composition

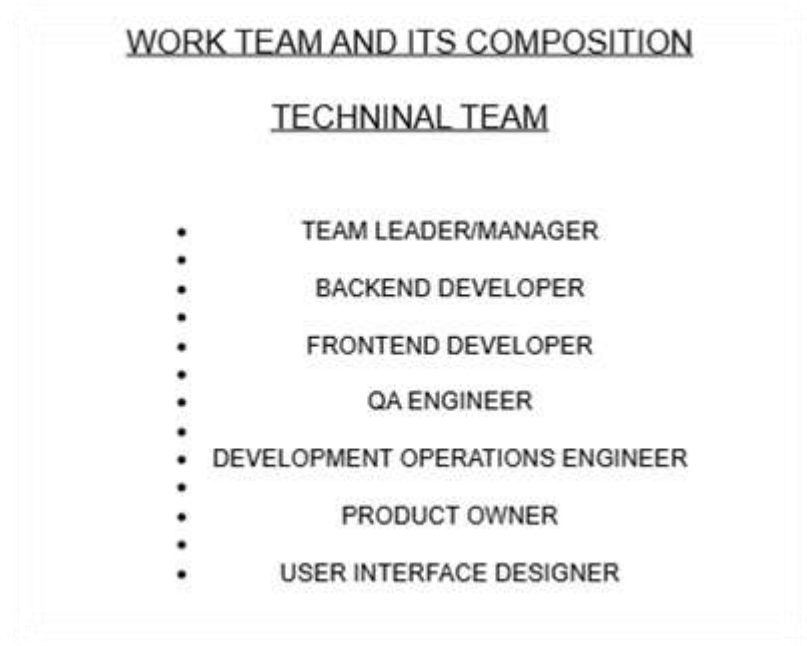


Figure 3

2.5. Working Relationship among team members/ other staffs

Cultivating positive workplace relationships is paramount to guarantee the prosperity of individuals and organizations. These relationships are pivotal in determining employees' job satisfaction and career advancement prospects. By acquiring a more profound comprehension of these relationships, individuals can revolutionize their professional endeavours and substantially contribute to their organization's triumph. Managing relationships with colleagues in the workplace is crucial for individuals and organizations. To achieve this, organizations must analyse the diverse types of workplace relationships.

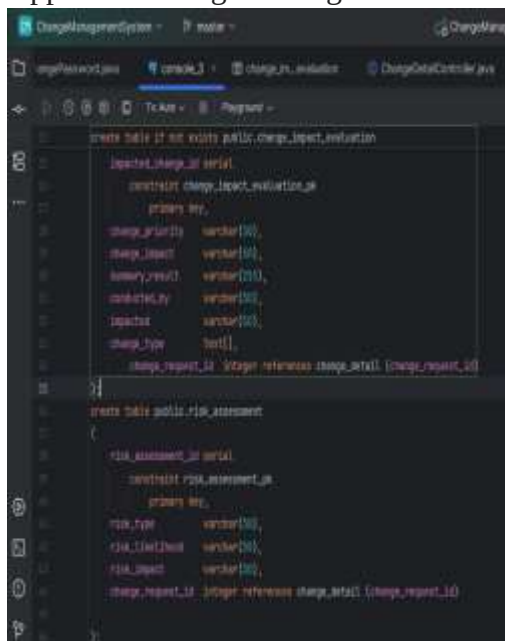
- **Colleague Relationships:** Effective collaboration and support among colleagues are essential for a positive workplace culture. Mutual respect, trust, and effective communication form the foundation of these healthy work relationships. Supportive colleague relationships result in better teamwork, harmony, and cooperation in the organization.
- **Supervisor-Subordinate Relationships:** For a workplace to thrive and for employees to progress in their professional trajectories, the interaction between individuals in positions of authority and their subordinates is paramount. Building a healthy work relationship with your supervisor helps you understand their expectations, receive feedback, and gain support in achieving your professional goals.
- **Cross-Functional Relationships:** Collaboration between different teams is crucial for achieving success in projects. Cross-functional relationships involve individuals from different departments working collectively toward shared objectives. Such healthy relationships foster the exchange of ideas and expertise throughout the organization, resulting in enhanced project outcomes and the emergence of innovative solutions.
- **Remote Work Relationships:** Remote work relationships are an important part of the present work culture and refer to professional relationships between individuals working from different locations. These relationships are possible through digital communication tools and involve virtual collaboration, flexible schedules, and a dependence on technology.

3.0. CHAPTER THREE: EVALUATION ON FIELD ATTACHMENT

The evaluation on field attachment derives in depth work flow from introduction of backend programming unto the coding, designing and implementation, debugging and testing an internship mini project.

3.0.1 Introduction to backend Programming

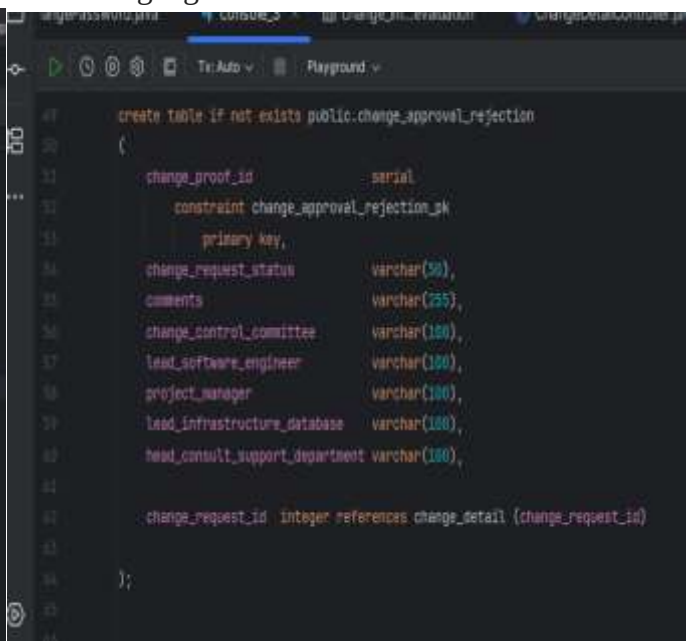
Backend development is the skill attained for one to design, development, test and implement software applications. It requires for one to undertake specific tools and mechanism of frameworks and therefore, during this internship were guided through the following requirements; Structured Query Language (SQL), Spring Boot Framework, DataGrip, IntelliJ, Application Programming Interface and Java Language and Postman



```
create table if not exists public.change_request_evaluation
(
    request_id serial,
    constraint change_request_evaluation_pk
        primary key,
    change_request_id integer references change_detail (change_request_id),
    change_request_status varchar(50),
    comments varchar(255),
    implemented_by varchar(50),
    signature varchar(100),
    change_request_id integer references change_detail (change_request_id)
);

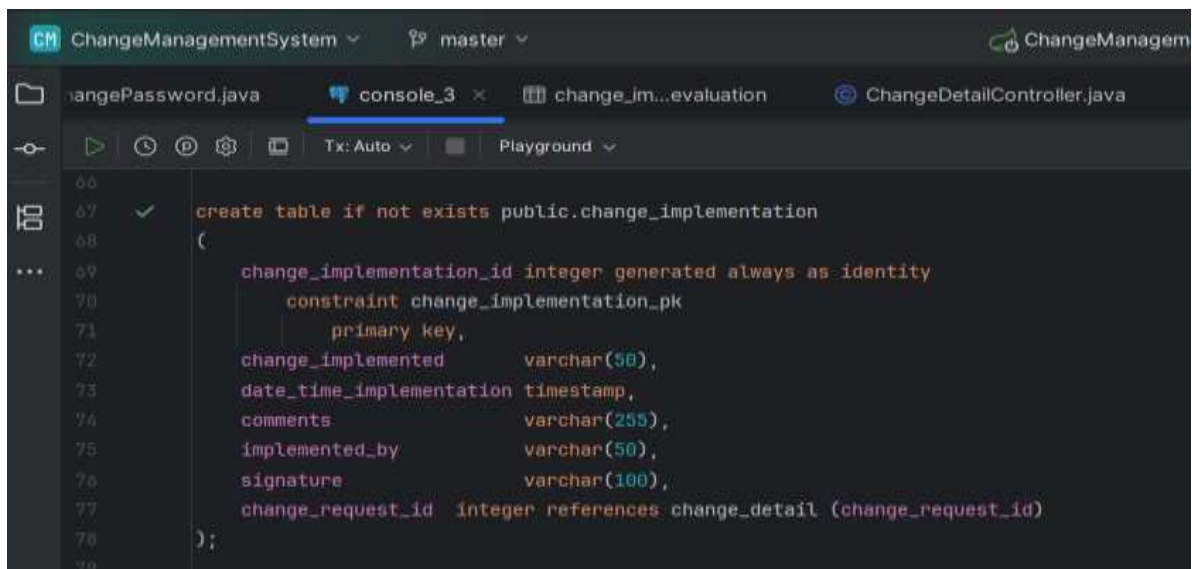
create table public.risk_assessment
(
    risk_assessment_id serial,
    constraint risk_assessment_pk
        primary key,
    risk_type varchar(50),
    risk_title varchar(100),
    risk_impact varchar(100),
    change_request_id integer references change_detail (change_request_id)
);
```

Figure 1



```
create table if not exists public.change_approval_rejection
(
    change_approval_id serial,
    constraint change_approval_rejection_pk
        primary key,
    change_request_status varchar(50),
    comments varchar(255),
    change_control_committee varchar(100),
    lead_software_engineer varchar(100),
    project_manager varchar(100),
    lead_infrastructure_database varchar(100),
    head_consult_support_department varchar(100),
    change_request_id integer references change_detail (change_request_id)
);
```

Figure 2



```
create table if not exists public.change_implementation
(
    change_implementation_id integer generated always as identity,
    constraint change_implementation_pk
        primary key,
    change_implemented varchar(50),
    date_time_implementation timestamp,
    comments varchar(255),
    implemented_by varchar(50),
    signature varchar(100),
    change_request_id integer references change_detail (change_request_id)
);
```

Figure 4

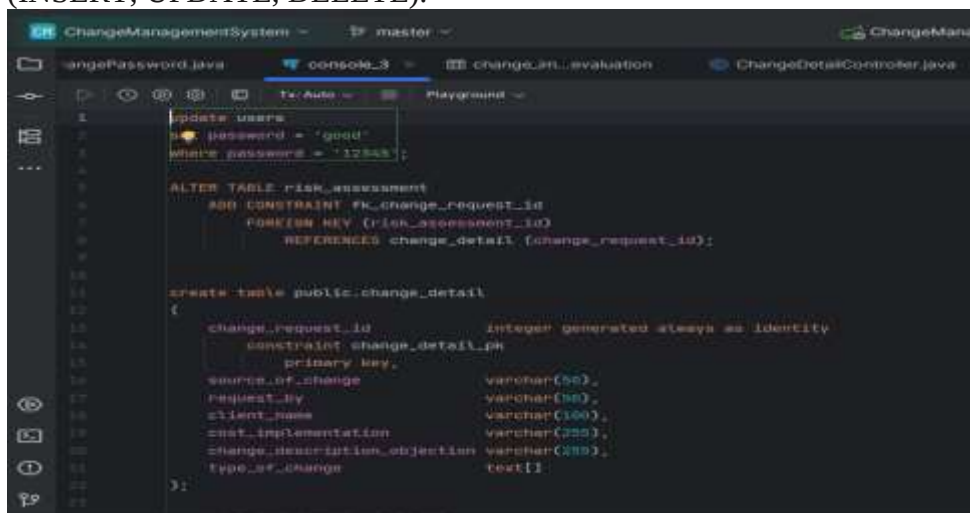
3.0.2. Structured Query Language

According to Tpoint Tech (2025), SQL is Structured Query Language used to manage data in a relational database system. This tag refers explicitly to the ISO/ANSI SQL standard; not to any specific implementation of that standard. Structured Query Language (SQL) is a special-purpose programming language designed for managing data held in a Relational Database Management System (RDBMS). SQL-like languages can also be used in Relational Data Stream Management Systems (RDSMS), or in "not-only SQL"(NoSQL) databases. SQL comprises of 3 major sub-languages:

- Data Definition Language (DDL): to create and modify the structure of the database;
- Data Manipulation Language (DML): to perform Read, Insert, Update and Delete operations on the data of the database;
- Data Control Language (DCL): to control the access of the data stored in the database.

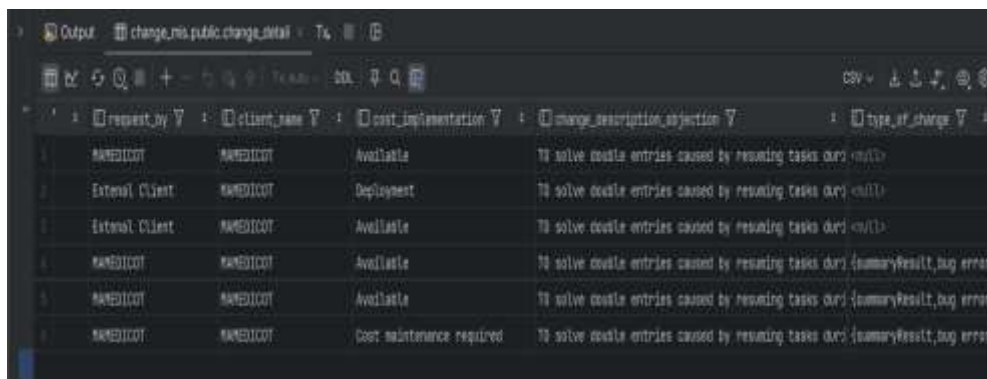
The core DML operations are Create, Read, Update and Delete (CRUD for short) which are performed by the statements INSERT, SELECT, UPDATE and DELETE.

There is also a (recently added) MERGE statement which can perform all 3 write operations (INSERT, UPDATE, DELETE).



```
1 update users
2   password = 'good'
3   where password = '12345';
4
5 ALTER TABLE risk_assessment
6 ADD CONSTRAINT fk_change_request_id
7 FOREIGN KEY (risk_assessment_id)
8 REFERENCES change_detail (change_request_id);
9
10
11 create table public.change_detail
12 (
13     change_request_id integer generated always as identity
14     constraint change_detail_pk
15     primary key,
16     source_of_change varchar(50),
17     request_by varchar(100),
18     client_name varchar(100),
19     cost_implementation varchar(255),
20     change_description_object varchar(255),
21     type_of_change text[]
22 );
```

Figure 5



request_by	client_name	cost_implementation	change_description_object	type_of_change
MANAGEDCOT	MANAGEDCOT	Available	To solve double entries caused by resuming tasks dco <null>	
External Client	MANAGEDCOT	Deployment	To solve double entries caused by resuming tasks dco <null>	
External Client	MANAGEDCOT	Available	To solve double entries caused by resuming tasks dco <null>	
MANAGEDCOT	MANAGEDCOT	Available	To solve double entries caused by resuming tasks dco {summaryResult,bug error}	
MANAGEDCOT	MANAGEDCOT	Available	To solve double entries caused by resuming tasks dco {summaryResult,bug error}	
MANAGEDCOT	MANAGEDCOT	Cost maintenance required	To solve double entries caused by resuming tasks dco {summaryResult,bug error}	

Figure 6

3.0.3. Java Language

According to Hamedani (2024), Java is a robust and versatile language, perfect for backend development. Known for its performance and scalability, Java is a top choice for building large-scale, enterprise-level applications.

According to Hamedani (2024), Essential Concepts are:

- Basics: Variables, primitive and reference types, constants, casting, arrays, arithmetic expressions
- Control Flow: Comparison operators, logical operators, ternary operator, if/else, switch/case, for, foreach, while, do..while, break, continue
- Object-oriented Programming: Classes, objects, constructors, method overloading, static members, inheritance, access modifiers, method overriding, upcasting/downcasting, polymorphism, abstract classes, final classes, interfaces
- Exceptions: Exceptions hierarchy, try/catch/finally, throw, custom exceptions
- Generics: Generic classes/methods/interfaces, constraints, type erasure, type parameters
- Collections Framework: Iterable, Iterator, Collection, List, Comparable, Comparator, Queue, Set, Map
- Functional interfaces: Lambda expressions, Consumer, Supplier, Function, Predicate, BinaryOperator, UnaryOperator

3.0.4. Spring Boot Framework

According to Walls, C. (2016), Spring is a framework and is written in the Java language. It provides lots of modules, such as Spring Data, Spring Security, Spring Cloud, Spring Web, and so on. It is popular for building enterprise applications. Initially, it was looked at as a Java Enterprise Edition (EE) alternative. Spring supports Dependency Injection (Inversion of Control) and Aspect-Oriented Programming out of the box as its core. Apart from Java, Spring also supports other JVM languages such as Groovy and Kotlin. The principle and design patterns – Inversion of Control (IoC), Dependency Injection (DI), and Aspect-Oriented Programming (AOP).

According to Walls, C. (2016), Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can “just run”. It takes an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration. You can use Spring Boot to create Java applications that can be started using java -jar or more traditional war deployments. It also provides a command line tool that runs “spring scripts”. primary goals are:

- Provide a radically faster and widely accessible getting started experience for all Spring development.
- Be opinionated out of the box, but get out of the way quickly as requirements start to diverge from the defaults.
- Provide a range of non-functional features that are common to large classes of projects (e.g., embedded servers, security, metrics, health checks, externalized configuration).
- Absolutely no code generation and no requirement for XML configuration.

According to Walls, C. (2016), Spring Boot brings a great deal of magic to Spring application development. But

there are four core tricks that it performs:

- Automatic configuration—Spring Boot can automatically provide configuration for application functionality common to many Spring applications.
- Starter dependencies—You tell Spring Boot what kind of functionality you need, and it will ensure that the libraries needed are added to the build.
- The command-line interface—This optional feature of Spring Boot lets you write complete applications with just application code, but no need for a traditional project build.
- The Actuator—Gives insight into what's going on inside of a running Spring Boot application.

Spring Boot doesn't implement any enterprise Java specifications such as JPA or JMS. It does support several enterprise Java specifications, but it does so by automatically configuring beans in Spring that support those features. For instance, Spring Boot doesn't implement JPA, but it does support JPA by auto-configuring the appropriate beans for a JPA implementation (such as Hibernate).

Finally, Spring Boot doesn't employ any form of code generation to accomplish its magic. Instead, it leverages conditional configuration features from Spring 4, along with transitive dependency resolution offered by Maven and Gradle, to automatically configure beans in the Spring application context.

3.0.5. Postman

Postman is an Application Programming Interface (API) testing tool. API acts like an interface between a couple of applications and establishes a connection between them (GeeksforGeeks, 2024).

An API is a collection of agreements, functions, and tools that an application can provide to its users for successful communication with another application. We require an API whenever we access an application like checking news over the phone, Facebook (GeeksforGeeks, 2024).

According GeeksforGeeks, (2024), Postman was designed in the year 2012 by software developer and entrepreneur Abhinav Asthana to make API development and testing straightforward. It is a tool for testing the software of an API. It can be used to design, document, verify, create, and change APIs.

Postman has the feature of sending and observing the Hypertext Transfer Protocol (HTTP) requests and responses. It has a graphical user interface (GUI) and can be used in platforms like Linux, Windows and Mac. It can build multiple HTTP requests – POST, PUT, GET, PATCH and translate them to code.

- Postman has a huge user base and has become a very popular tool because of the reasons listed below:
- Postman comes without any licensing cost and is suitable for use for the teams with any capacity. It can be installed as a standalone application.
- Postman allows easy maintenance of test suites with the help of collections. Users can make a collection of API calls which can have varied requests and sub-folders.
- Postman is capable of building multiple API calls like SOAP, REST, and HTTP.
- Postman can be used for test development by addition of checkpoints to HTTP response codes and other parameters.

- Postman can be integrated with the continuous integration and either continuous delivery or continuous deployment (CI/CD) pipeline.
- Postman has big community support. The Postman console allows debugging test steps.
- Postman gives the option to import/export Environments and Collections, enabling easy sharing of files.

Postman can be installed in operating systems like Mac, Windows and Linux. It is basically an independent application which can be installed in the following ways:

- Postman can be installed from the Chrome Extension (will be available only in Chrome browser).

3.0.6. DataGrip

According to RisingWave (2024), DataGrip is an IDE designed to make the process of analysing any amount of data easier and more efficient. It enables users to access multiple databases in one interface, streamlining the process of analysing large amounts of data quickly and accurately. It also provides features that allow developers to create applications faster with less code, making it easy for any user regardless of their experience level to get started quickly and accurately. It is designed to simplify this process, making it easier and more efficient for data scientists, analysts, and anyone working with data.

developers. According to RisingWave (2024), DataGrip is a database IDE tailored to suit the specific needs of professional SQL Widely used by Enterprises , it simplifies the process of analysing large amounts of data quickly and accurately. This versatile tool supports various databases like MySQL, PostgreSQL, SQL Server, Oracle, and more. With features such as a database explorer, SQL editor, and query builder, it provides intelligent coding assistance for editing SQL code efficiently. With an intuitive editor offering intelligent coding assistance, including auto-completion and query profiling capabilities, it enhances the efficiency and accuracy of SQL code editing.

The purposes of DataGrip (RisingWave 2024):

- DataGrip stands out as a comprehensive IDE designed to streamline database management tasks for SQL developers. Its primary objective is to provide a unified platform for accessing and manipulating various databases seamlessly. By offering advanced features like context-sensitive code completion and query execution modes, empowers users to work with multiple databases efficiently.
- The strength of DataGrip lies in its extensive support for numerous databases, ensuring compatibility with popular systems like MySQL, PostgreSQL, Oracle, and more. This broad database coverage enables SQL developers to work across different platforms without encountering compatibility issues or limitations.
- The standout features are its multi-engine support, allowing users to connect to diverse database systems seamlessly. Whether it's MySQL, PostgreSQL, or Microsoft SQL Server, facilitates easy integration with various engines through its intuitive interface.

The key features of DataGrip:

- The Context-Sensitive Code Completion feature is a game-changer for developers. By analysing the context of the code being written, intelligently suggests relevant code snippets, table names, and even column names. This functionality significantly accelerates the coding process by reducing manual input errors and streamlining the development workflow. For developers, the benefits are immense. It not only saves time by offering quick suggestions but also improves code accuracy by minimizing typos and syntax errors.
- Developers have the flexibility to execute queries in various modes tailored to their specific needs. Whether it's running queries for data retrieval, modification, or analysis, provides different execution modes to optimize performance and enhance result accuracy. By understanding these modes, developers can choose the most suitable option based on their query requirements.
- The SQL Log feature acts as a comprehensive audit trail for all executed queries. By maintaining a detailed log of query history, developers can track changes made to databases over time, identify performance bottlenecks, and troubleshoot issues effectively. This visibility into query execution is crucial for maintaining database integrity and optimizing system performance.

3.0.7. IntelliJ

According to TechSpot (2025), IntelliJ IDE is a powerful integrated development environment primarily designed for Java development with extensive support for other languages and frameworks.

IntelliJ IDEA comes with one of the most powerful code editors in the industry. It understands the ins and outs of your code thanks to initial indexing, which allows it to detect errors on the fly, suggest code completion options with precise context awareness, perform safe refactoring, and much more.

IntelliJ IDEA uses machine learning to ensure the most relevant suggestion is at the top of the list.

Features (TechSpot, 2025):

- Full line code completion: powered by an advanced deep learning model, this feature enhances coding efficiency by predicting and suggesting entire line of code based on contextual analysis. It runs entirely on your device, ensuring your data remains secure without any external server communication.
- AI Assistant General Availability: AI Assistant comes with useful functionality that can simplify your daily tasks. It offers integrated AI chat can do things like automatically write documentation comments for you, suggest names, generate commit messages, and more.
- Language Injections: IntelliJ IDEA lets you work with elements of code in other languages embedded in your code. Won't notice a difference between your experience working with the core Java code and experience with the code that you have injected. Provide code highlighting, completion, error detection, and other coding assistance features for embedded code fragments.
- Intelligent code completion: Provides context sensitive code completion, offering suggestions that are only valid for the current position. Don't need to use any shortcuts or configure additional settings; code completion will start as soon as you begin typing your code in the editor.

3.0.8. Restful API

Representational state transfer (REST) is a software architectural style that defines a set of constraints to be used for creating Web services. RESTful Web services allow the requesting systems to access and manipulate textual representations of Web resources by using a uniform and predefined set of stateless operations (Kandwal, 2020).

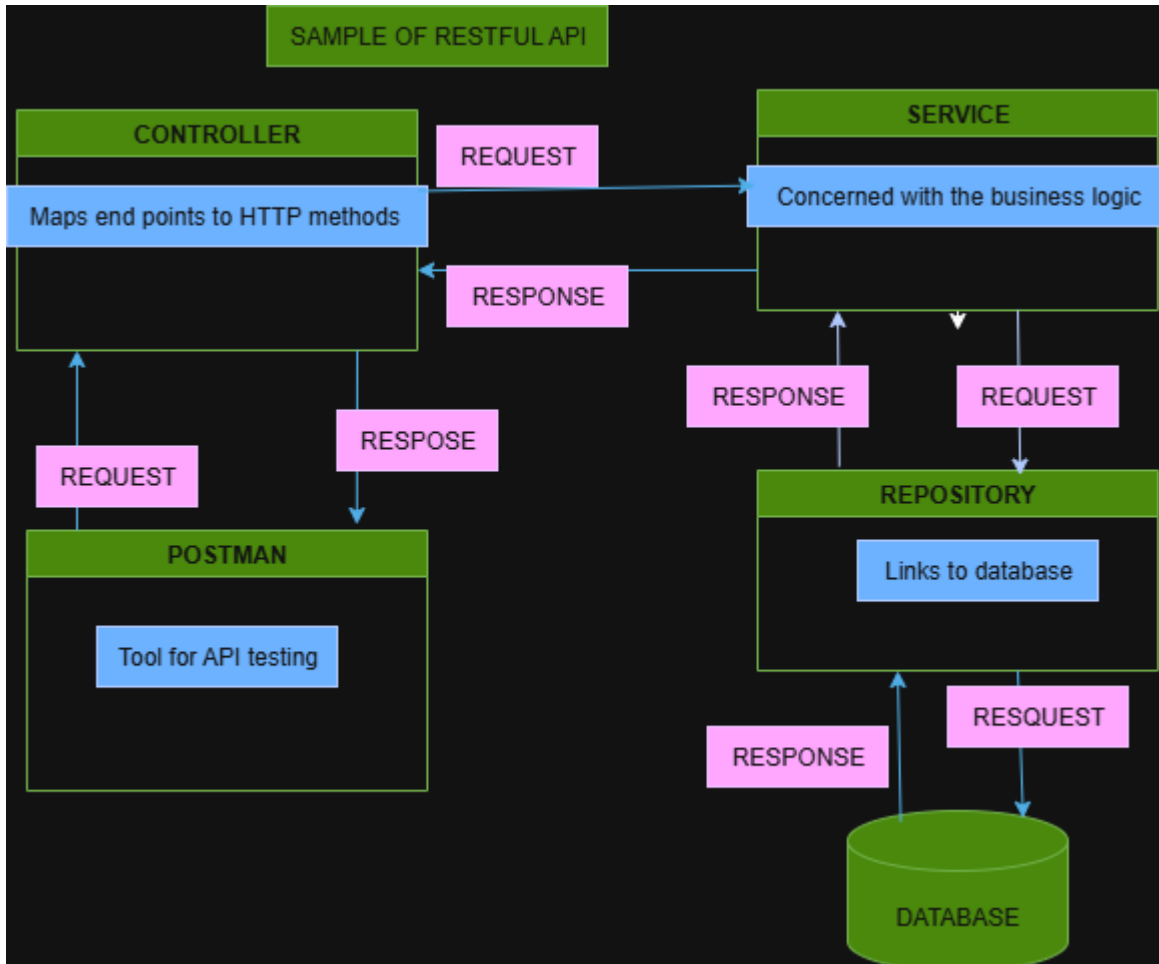


Figure 7

REST is an acronym for REpresentational State Transfer and an architectural style for distributed hypermedia systems (Gupta, 2025).

According to Gupta (2025), REST is not a protocol or a standard, it is an architectural style. During the development phase, API developers can implement REST in a variety of ways. REST also has its guiding principles and constraints. These principles must be satisfied if a service interface is to be referred to as RESTful. A Web API (or Web Service) conforming to the REST architectural style is called a REST API (or RESTful API).

REST is based on some constraints and principles that promote simplicity, scalability, and statelessness in the design. The six guiding principles or constraints of the RESTful architecture are:

- Uniform Interface: By applying the principle of generality to the components interface, we can simplify the overall system architecture and improve the visibility of interactions. Multiple architectural constraints help in obtaining a uniform interface and guiding the behaviors of components.
- Client-Server: The client-server design pattern enforces the separation of concerns, which helps the client and the server components evolve independently.
- Stateless: This mandates that each request from the client to the server must contain all of the information necessary to understand and complete the request.
- Cacheable: The cacheable constraint requires that a response should implicitly or explicitly label itself as cacheable or non-cacheable.
- Layered System: The layered system style allows an architecture to be composed of hierarchical layers by constraining component behaviors. In a layered system, each component cannot see beyond the immediate layer they are interacting with.
- Code on Demand: REST also allows client functionality to be extended by downloading and executing code in the form of applets or scripts.

According to Gupta (2025), The key abstraction of information in REST is a resource. Any information that we can name can be a resource. For example, a REST resource can be a document or image, a temporal service, a collection of other resources, or a non-virtual object (e.g., a person). The state of the resource at any particular time is known as the resource representation. The resource representations consist of the data, the meta data describing the data and the hypermedia links that can help the clients transition to the next desired state.

Rest API key points (Massey, L. (n.d.)):

- RESTful APIs typically use HTTP methods like GET, POST, PUT, DELETE, etc.
- They exchange data in JSON format, making it easy for humans and machines to read.
- A RESTful service responds to requests with:
- An HTTP status code (e.g., 200 OK, 404 Not Found)
- A body containing data, usually in JSON
- REST doesn't require a specific protocol, but most modern REST APIs use HTTP

The five primary HTTP methods are as follows:

Method	Description
GET	Read a specific resource (by an identifier) or a collection of resources.
PUT	Update/Replace a specific resource (by an identifier) or a collection of resources. Can also be used to create a resource or a collection of resources when the identifier is known before-hand.
PATCH	Modify a specific resource (by an identifier) or a collection of resources.
DELETE	Remove/Delete a specific resource by an identifier
POST	Create a new resource.

Table 3

According to Massey, L. (n.d.), Favour JSON support unless you're in a highly-standardized and regulated industry that requires XML, schema validation and namespaces. If you must provide both JSON and XML let consumers switch between formats using the HTTP, Accept header: application/json or application/xml.

Response status codes are part of the HTTP specification. There are quite a number of them to address the most common situations. In the spirit of having our RESTful services embrace the HTTP specification, our Web APIs should return relevant HTTP status codes.

According to Massey, L. (n.d.), Appropriate resource names provide context for a service request, increasing understandability of the API. Resources are viewed hierarchically via their URI names, offering consumers a friendly, easily-understood hierarchy of resources to leverage in their applications. The quick-hit rules for URL path (resource name) design:

- Use identifiers in your URLs instead of in the query-string. Using URL query-string parameters is fantastic for filtering, but not for resource names: Good: /users/12345
- Poor: /api?type=user&id=23
- Leverage the hierarchical nature of the URL to imply structure.
- Design for your clients, not for your data.
- Resource names should be nouns. Avoid verbs as resource names, to improve clarity. Use the HTTP methods to specify the verb portion of the request.
- Use plurals in URL segments to keep your API URIs consistent across all HTTP methods, using the collection metaphor: Recommended: /customers/33245/orders/8769/lineitems/1
- Not: /customer/33245/order/8769/lineitem/1
- Avoid using collection verbiage in URLs. For example 'customer_list' as a resource. Use pluralization to indicate the collection metaphor (e.g. customers vs. customer_list).
- Use lower-case in URL segments, separating words with underscores ('_') or hyphens ('-'). Some servers ignore case so it's best to be clear.
- Keep URLs as short as possible, with as few segments as makes sense.

3.1. Level of Accomplishment of duties and responsibilities assigned

DUTIES	RESPONSIBILITIES
Introduction to Java backend programming	Assisting to understand Background concept review
Introduction to Structured Query Language (SQL)	Hands on practical script writing
Introduction to Data Grip	Brief and precise explaining of concept
Introduction to Microsoft Teams	Download and installation
Introduction to Object Oriented Programming	Writing SQL script on Console
Introduction to Spring Boot	Account creating
Introduction to IntelliJ	Review on Object Oriented Programming concepts
Introduction to Application Programming Interface (API)	Precise concept explanation
	Write Spring Boot projects in IntelliJ
	Understanding Spring Boot framework
	Writing source code in Java language
	Writing Spring Boot projects with Java
	Consuming Application Programming Interface (API)

Figure 8

3.2. New knowledge and skills gained in each of the duties and responsibilities

DUTIES	RESPONSIBILITIES	KNOWLEDGE AND SKILLS
Introduction to Java backend programming	Assisting to understand Background concept review	Depth in Java backend programming
Introduction to Structured Query Language (SQL)	Hands on practical script writing	SQL concepts & Join Concepts View concept
Introduction to Data Grip	Brief and precise explaining of concept	Writing script of single tables
Introduction to Microsoft Teams	Download and installation	Writing scripts of multiple tables
Introduction to Object Oriented Programming	Writing SQL script on Console	Data Grip tool and Console interfacing
Introduction to Spring Boot	Account creating	Designing databases and Script understanding
Introduction to IntelliJ	Review on Object Oriented Programming concepts	Spring Boot framework
Introduction to Application Programming Interface (API)	Precise concept explanation	Spring Annotation and Spring JdbcTemplate
	Write Spring Boot projects in IntelliJ	In-depth understanding of Spring Boot
	Understanding Spring Boot framework	Rapid Application Development
	Writing source code in Java language	In-depth understanding Project development
	Writing Spring Boot projects with Java	Project structuring
	Consuming Application Programming Interface (API)	Consuming internal Application Programming Interface (API)
		Consuming external Application Programming Interface

Figure 9

3.3. Most interesting experiences

1. Learning Java backend programming.
2. Learning from experienced programming team.
3. Hands on work doing.
4. Open minded experiencing.
5. Supervisor to Internee communication.
6. Mentoring and guidance from various department.
7. Free minded expression.
8. High co-operation team work.
9. Open willingness of every member.
10. High professional skills.

3.4. Relatedness of University's taught programs to the Field of work

Below are the University's taught programs:

1. Java language programming.
2. Communication skills.
3. Problem solving.
4. Ethics and integrity.
5. Database Management.
6. Project Management and development.

3.5. Challenges faced and how managed

Work related challenges	How managed
Task completion	By completing particular task assigned.
Task design and development.	By reading and researching.
Misunderstanding tasks.	By consulting the supervisor.
Failure to understand tasks.	By asking assistance and explanation from supervisor.

Table 4

Organizational challenges	How managed
Time management.	By reporting with regulated time frame.
Cooperation with others.	By creating a friendly environment.
Understanding organizational structure.	By communicating with others even when from other departments.

Table 5

Individual perspective challenges	How managed
Naïve in communication	By daily communication practice.
Naïve open expression	By daily open-minded practice.
Poor explaining concepts	By regular practices for brief and precise explanations
Hygiene	By practicing company hygiene environment
Time management	By making sure report with early time range.
Project work making	By making sure completion of particular task.

Table 6

3.6. Benefits derived from Field Attachment

- Learning Java backend programming.
- Learning from experienced programming team.
- Hands on work doing.
- Open minded experience.
- Supervisor to Internee communication.
- Mentoring and guidance from various department.
- Free minded expression.
- High co-operation team work.
- Open willingness of every member.
- High professional skills.
- Improved programming skills.
- Good communication skills.

3.7. Adequacy in University's preparing the student for Field Attachment

The University take the perspective, asking what colleges do to make themselves 'student ready,' including meeting the needs of students academically, social-emotionally, and financially. These supports are critical for increasing a student's sense of well-being and belonging for Field Attachment which increases persistence towards academic goals and matriculation (Caldwell et al., 2021).

Caldwell et al. (2021), The field of education has seen an increased focus on ensuring that students are college and career ready has written extensively on college and career readiness, and asserts that being college ready means ensuring students took rigorous courses and met specific metrics on standardized tests, identified analysis, interpretation, precision and accuracy, problem solving, and reasoning as essential cognitive and metacognitive skills for college students, also noted that while writing is a skill closely associated with college success, students must have knowledge of the big ideas of each content area. Additionally, students must have academic self-management behaviors, including time management, effective study skills, and persistence. Lastly, contextual skills such as how to navigate college choice, admissions, and financial aid and knowledge are necessary to navigate institutional systems.

Oconnell (2025), Preparing students for the future is a challenging task, especially when considering the unpredictable nature of the job market and the world economy. To prepare students for the future of work, universities must provide more than just academic instruction. This includes offering internships, cooperative education programs, and courses focused on practical skills such as data analytics, computer skills, technological advancements concerning project management and machine learning. Additionally, universities can offer career counselling and digital portals to help students navigate the evolving employment landscape.

University provides remote teaching and resources, allowing students to develop digital skills and flexible study habits. To foster adaptability and creative thinking, educators should focus on project-based learning and encourage problem-solving activities. Furthermore, it is essential to provide students with real-world experiences and discussions around topics like artificial intelligence and its ethical implications.

3.8. Preparedness of the Agency to receive and manage Students for Field Attachment

According Parkes, M., Stein, S. and Reading, C. (2015), Field Attachment preparedness from the perspectives of university teachers (staff) and students across different disciplines and institutions. The term Field Attachment preparedness is complex and involves several dimensions, such as ‘being informed, making the right choices, having realistic expectations and being motivated’. Some scholars argue that students enter internship with weaker basic disciplinary skills than needed (for example, statistical skills for psychology studies), while others argue that students show weak generic writing and reading skills). Still others point to the importance of behavioural characteristics, such as academic perseverance.

According Parkes, M., Stein, S. and Reading, C. (2015), Preparedness is conceptualised as an overarching term comprising reading and writing skills, basic disciplinary skills and academic working skills, all of them preconditions for further academic development. The focus is on students’ and university teachers’ experiences related to these dimensions. Academic staff are crucial for student learning but are also guardians of quality in their disciplines, setting the standards for what counts as good performance. Students are also important informants as they are in a process of grasping what is required from them in their formative time as students in for Field Attachment.

3.9. Career Motivation

Career motivation is the desire to exert effort to enhance career goals (IResearchNet, n.d.). It is a multidimensional construct that combines elements of needs, interests, and personality characteristics that reflect the stimulus, direction, and persistence of career-related behaviours. According to IResearchNet (n.d.), Career motivation is organized into three domains. Career insight is the stimulus or energizing component. This is people’s ability to be realistic about themselves and their careers. People who are high in career insight have an accurate understanding of their strengths and weaknesses and set clear career goals as below;

- Understand the deeper purpose behind your work.
- Reflect on how your role contributes to your personal goals or helps others.
- Specific, Measurable, Achievable, Relevant, Time-bound.
- Break big goals into smaller milestones to track progress and stay focused.
- Acknowledge every achievement, no matter how minor.
- Take courses, attend workshops, or explore new tools.
- Continuous growth keeps your work fresh and boosts confidence.
- Surround yourself with mentors, peers, and professionals who uplift you.
- Share goals and challenges—it helps you stay accountable and inspired.
- Focus on what’s going well rather than what’s lacking.
- Keep a gratitude or success journal to reflect on progress.
- Try new approaches to routine tasks.
- Creativity can reignite excitement and help you see things from a fresh perspective.
- Avoid burnout by setting boundaries and taking breaks.
- Recharge regularly to return with renewed energy.

4.0. CHAPTER FOUR: CONCLUSIONS AND RECOMMENDATIONS

4.1. Conclusions

Educational transformation is not an option; it is imperative for institutions seeking to prepare students for the challenges and opportunities of the future. Innovative approaches, awareness of key trends, adaptability, and a commitment to lifelong learning are the cornerstones of educational institutions that successfully navigate this transformative journey. By examining successful case studies, institutions can draw inspiration and practical insights as they embark on their path toward educational transformation.

In an era marked by rapid technological advancements and evolving societal needs, educational institutions must continually adapt to ensure they remain relevant and effective. This transformative landscape calls for practical methods and strategies drawn from the literature to prepare students effectively for the ever-changing job market (UNESCO's Action in Education, 2022). In this article, the researcher explores a range of strategies rooted in research and practice. The researcher delves into how colleges and universities can navigate the evolving demands of education and address gaps regarding the effectiveness of certain preparation methods. Moreover, the researcher examines the role of extracurricular activities and internships, the impact of the pandemic on the job market, the implications of remote learning, and the vital aspect of students' mental health and well being in preparing them for future jobs.

The transformation of the technologies, accelerating certain trends and amplifying the demand for specific skills. In this report, the author explores how the Field Attachment has reshaped the landscape and delves into the implications for education. However, it is imperative to recap the main findings and insights from this report, underscore the critical importance of preparing students for the Field Attachment and connect these insights to the specific context. Throughout this report, the author has navigated the evolving terrain of Field Attachment, recognizing the fundamental role of colleges and universities in equipping students with the skills needed for a rapidly changing market. The findings emphasize the significance of fostering adaptability, critical thinking, empathy, integrity, optimism, proactivity, and resilience (Emma, 2019). These skills serve as the foundation upon which students can survive and thrive in the face of uncertainty and emerging challenges.

By integrating future skills into its curricula, providing robust mental health support to students, and embracing technological advancements (Microsoft Education Team, 2023; International Labour Organization, 2020), University can empower its students to thrive in the ever-evolving world of work. It serves as a compass guiding Institutions towards effective strategies that empower students thrive in the global market while contributing to the economic growth. As all move forward, let us remember that the future belongs to those who are prepared, adaptable, and equipped with the skills to shape the world of tomorrow, colleges and universities have a crucial role to play in this journey.

4.2. Recommendations

The University should provide more than just academic instruction to prepare students for the future of work. This includes offering internships, cooperative education programs, and courses focused on practical skills such as problem solving, logical context, data analytics and machine learning. Additionally, University can offer career counselling and digital portals to help students navigate the evolving employment landscape.

The University should provide remote teaching and resources, allowing students to develop digital skills and flexible study habits to foster adaptability and creative thinking, educators should focus on project-based learning and encourage problem-solving activities. Furthermore, it is essential to provide students with real-world experiences and discussions around topics like artificial intelligence and its ethical implications.

The students should emphasize the significance of fostering adaptability, critical thinking, empathy, integrity, optimism, proactivity, and resilience skills to serve as the foundation upon which students can survive and thrive in the face of uncertainty and emerging challenges.

The following are generalized recommendations:

- **Robust Research and Evaluation:** Educational institutions should prioritize rigorous research and evaluation of their programs and strategies (Bakhshi et al., 2017). This includes assessing the effectiveness of preparation methods and continuously refining them based on empirical evidence.
- **Integration of Future Skills:** Curricula should be updated to systematically integrate future skills such as digital literacy, problem-solving, and emotional intelligence (Marr, n.d.). These skills should be embedded throughout the educational journey.
- **Mental Health Support:** Mental health and well-being services should be readily available to students, both in-person and online (Marr, n.d.). These services should be destigmatized and integrated into the educational experience.
- **Effective Preparation by Educational Institutions:** Colleges and universities play a pivotal role in shaping the preparedness of students for the evolving market.
- **Extracurricular Activities and Internships:** Extracurricular activities and internships offer students invaluable opportunities to apply their knowledge in real-world settings and universities can maximize the impact of these experiences in preparing students for their future careers.
- **Adapting to Future Challenges:** Educational institutions must be agile and adaptive to meet future challenges. Key strategies for adaptation include: Flexibility, Data-Driven Decision-Making and Interdisciplinary Approaches.
- **Innovative Approaches and Strategies:** Innovation in education is not merely a buzzword; it is an imperative. The literature is replete with innovative approaches and strategies that institutions can employ to enhance the learning experience. These include: Blended Learning Models, Personalized Learning, Competency-Based Learning and Experiential Education.

5.0. CHAPTER FIVE: REFERENCES

- Schildt, H. (2014) **Java: The Complete Reference**. 9th edn. New York: McGraw-Hill Education.
- PostgreSQL Global Development Group (2023) **PostgreSQL Documentation**. Available at: <https://www.postgresql.org/docs>.
- Walls, C. (2016) **Spring Boot in Action**. Shelter Island, NY: Manning Publications.
- Tpoint Tech (2025) **SQL Tutorial: Learn SQL for Beginners and Professionals**. Available at: <https://www.tpointtech.com/sql-tutorial>.
- Canepa, G. (2016) **PostgreSQL Database Handbook**. Magdeburg: University of Magdeburg. Available at: <http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf>.
- Sharma, S. (2021) **Modern API Development with Spring and Spring Boot**. Birmingham–Mumbai: Packt Publishing. Available at: https://download.bibis.ir/Books/Programming/java/2021/Modern-API-Development-with-Spring-and-Spring-Boot_by-Sourabh-Sharma_bibis.ir.pdf.
- GeeksforGeeks (2024) **Postman Tutorial**. Available at: <https://www.geeksforgeeks.org/software-testing/postman-tutorial>.
- W3Schools (2025) **SQL Tutorial**. Available at: <https://www.w3schools.com/sql>.
- GoalKicker (2018) **Spring Framework Notes for Professionals**. Compiled from Stack Overflow Documentation. Available at: <https://goalkicker.com/SpringFrameworkBook/SpringFrameworkNotesForProfessionals.pdf>.
- Spring.io (2025) **Building REST Services with Spring**. Available at: <https://spring.io/guides/tutorials/rest>.
- Chacon, S. and Straub, B. (2014) **Pro Git**. 2nd edn. New York: Apress. Available at: <https://www.dbooks.org/pro-git-1484200764/>.
- Parkes, M., Stein, S. and Reading, C. (2015) ‘Student preparedness for university e-learning environments’, **The Internet and Higher Education**, 25, pp. 1–10. Available at: https://www.researchgate.net/publication/268691014_Student_preparedness_for_university_e.
- DataCamp (2024) **GIT Push and Pull Tutorial**. Available at: <https://www.datacamp.com/tutorial/git-push-pull>.

KeyDifferences (no date) *Difference Between Duties and Responsibilities*. Available at: <https://keydifferences.com/difference-between-duties-and-responsibilities.html>.

restfulapi.net, 2024. *REST API Tutorial*. [online] Available at: <https://restfulapi.net/>.

Kandwal, V., 2020. *REST API Tutorial – REST Client, REST Service, and API Calls Explained with Code Examples*. [online] freeCodeCamp. Available at: <https://www.freecodecamp.org/news/rest-api-tutorial-rest-client-rest-service-and-api-calls-explained-with-code-examples/>.

Gupta, L., 2025. *What is REST? REST API Tutorial*. [online] restfulapi.net. Available at: <https://restfulapi.net/>.

RisingWave, 2024. *Datagrip: Your Ultimate Guide to Database Management*. [online] RisingWave. Available at: <https://risingwave.com/blog/datagrip-demystified-a-comprehensive-overview/>.

IResearchNet. (n.d.) *Career motivation*. [online] Career Development. Available at: <https://career.iresearchnet.com/career-development/career-motivation/>.

Massey, L. (n.d.) *What is REST? REST API Tutorial*. Available at: <https://restapitutorial.com/introduction/whatisrest>

Caldwell, K.W., Millis, C., Constant, T.N., Borg, P., Threatt-Morgan, K. and Burke, C.J.F. (2021) ‘Student readiness of colleges: A qualitative study’, *Journal of College Access*, 6(1), Article 5. Available at: <https://files.eric.ed.gov/fulltext/EJ1313687.pdf>.

Oconnell, L. (2025) ‘Preparing students for the future: The university’s role’, *ShunStudent*. Available at: <https://shunstudent.com/article/how-can-a-university-prepare-students-for-the-future>.

6.0. CHAPTER SIX: APPEDICES

6.1 INTERSHIP MINI PROJECT: CHANGE REQUEST MANAGEMENT

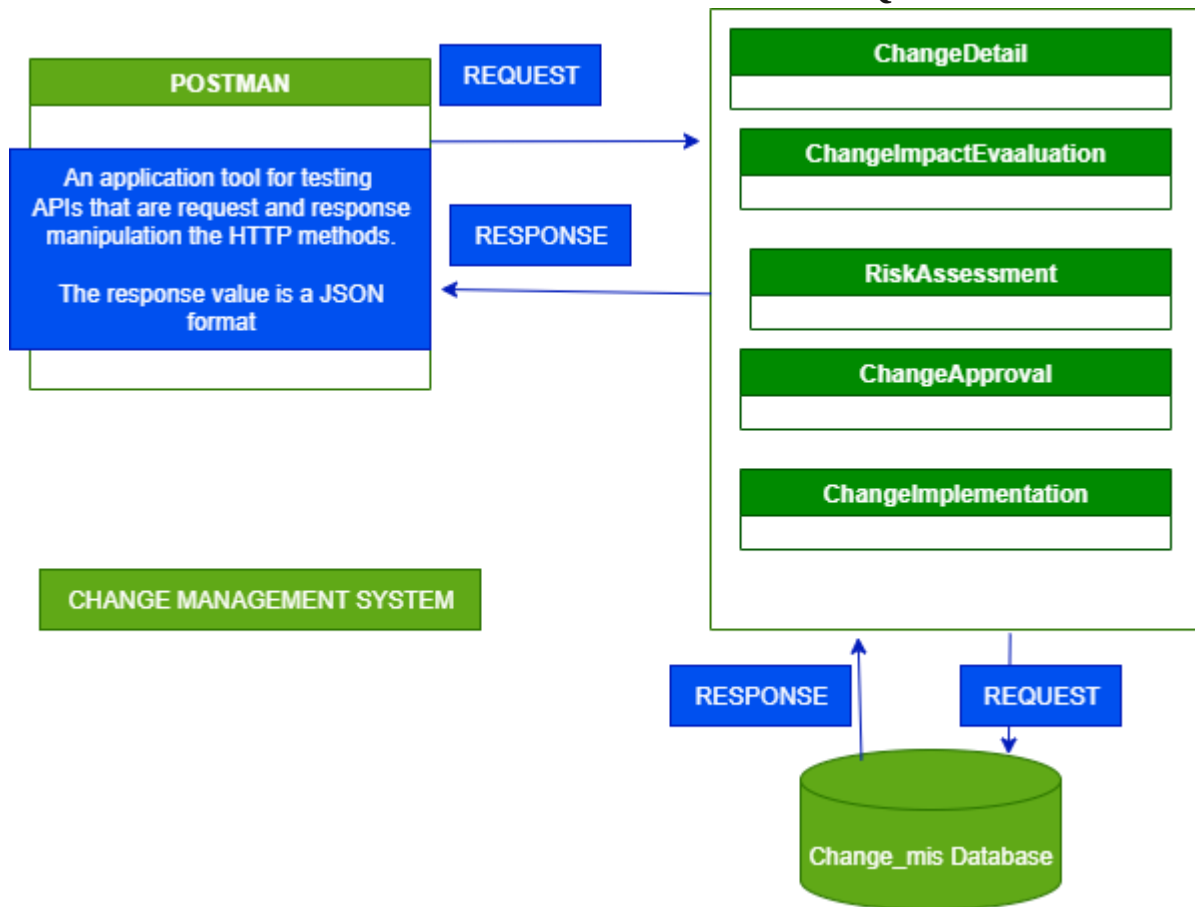


Figure 10

The Change Management System designed, implemented during the internship is an implication of the workflow concepts in which one makes a change request of either a new functional requirement, or creation of new modification about a part of system or application. In manual process, one documents the change request by filling the change request form comprising the first part of change detail part, risk assessment part and the change Impact evaluation part then proceeds to attain either the Approval or Reject from the technical teams. In addition, the technical team provide signatures in the hierarchal process in that when one signature is not filled, the form remains pending and when all signatures provided, the form appends to the implementation part. Therefore, with the design, coding and implementation of computerized internship mini project was attained as a demo system.

The buildup of the mini project was designed using the Spring Boot framework, Java language, Java Versions of 1.7 +, PostgreSQL, IntelliJ and Postman for backend programming to mainly create APIS (Application Programming Interfaces) to be consumed by the Frontend while the colleague worked on the Frontend using IntelliJ, TypeScript, HTML, CSS, JavaScript, Quasar framework, and WebStorm to create the Graphical User Interface.

The image below shows the mini project structure. The project was written in Java language using Spring Boot Framework structure and an IntelliJ tool. a couple of empty directories, it also includes the following:

- build.gradle—A Gradle build specification.
- Application.java—A class with a main () method to bootstrap the application.
- ApplicationTests.java—An empty JUnit test class instrumented to load a Spring application context using Spring Boot auto-configuration.
- application.properties—An empty properties file for you to add configuration properties to as you see fit.

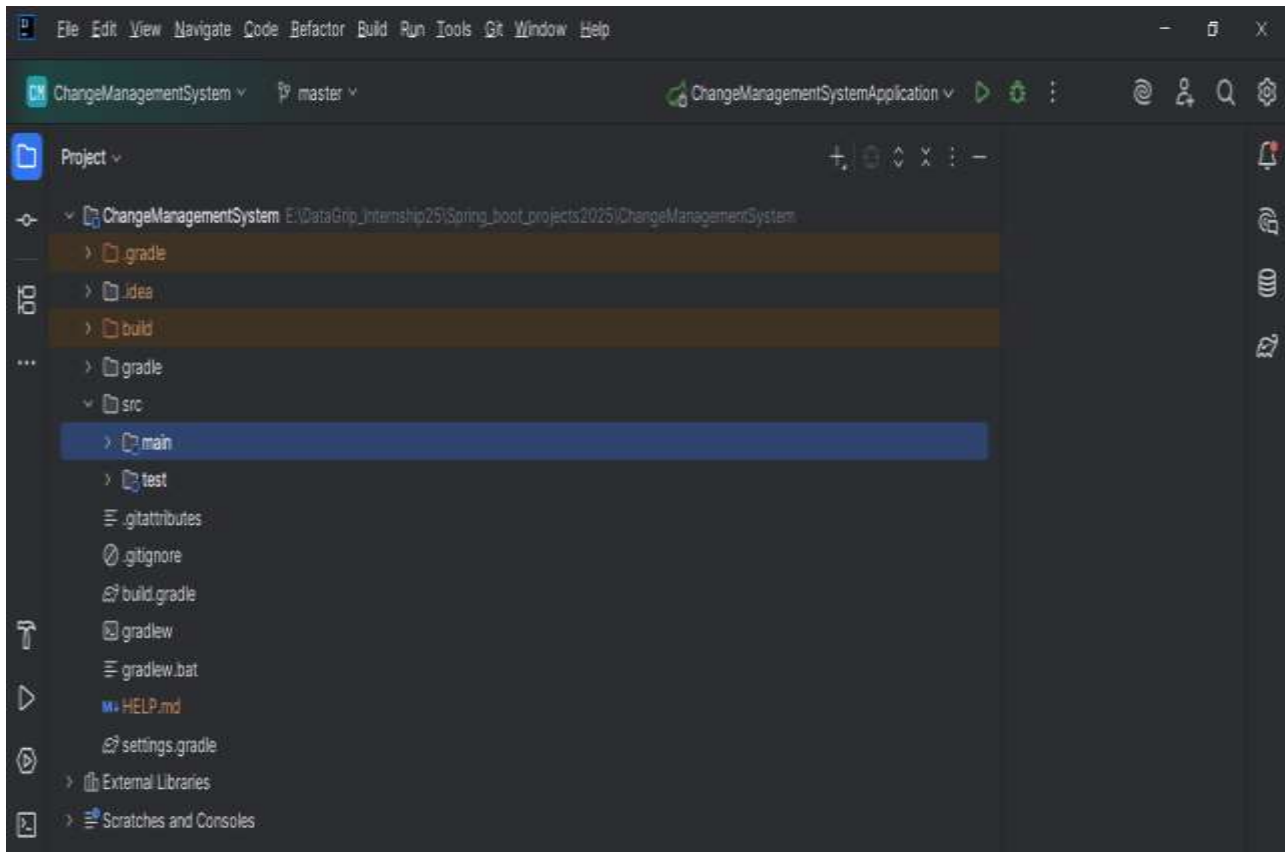


Figure 11

The diagram below shows the source (src) folder structure in which the main folder comprises of the section of creating modals, Java classes. The various folders in the main are as follows:

Controller: Contains Java classes that encapsulates the mapping of Http Request to methods of response class.

Modal: This is for create plain Old Java Objects.

Repository: This folder is for creating classes that connect or linkup to the database archives using the JdbcTemplate of the JDBC API.

Service: This is for creating classes for constituting business logic writing.

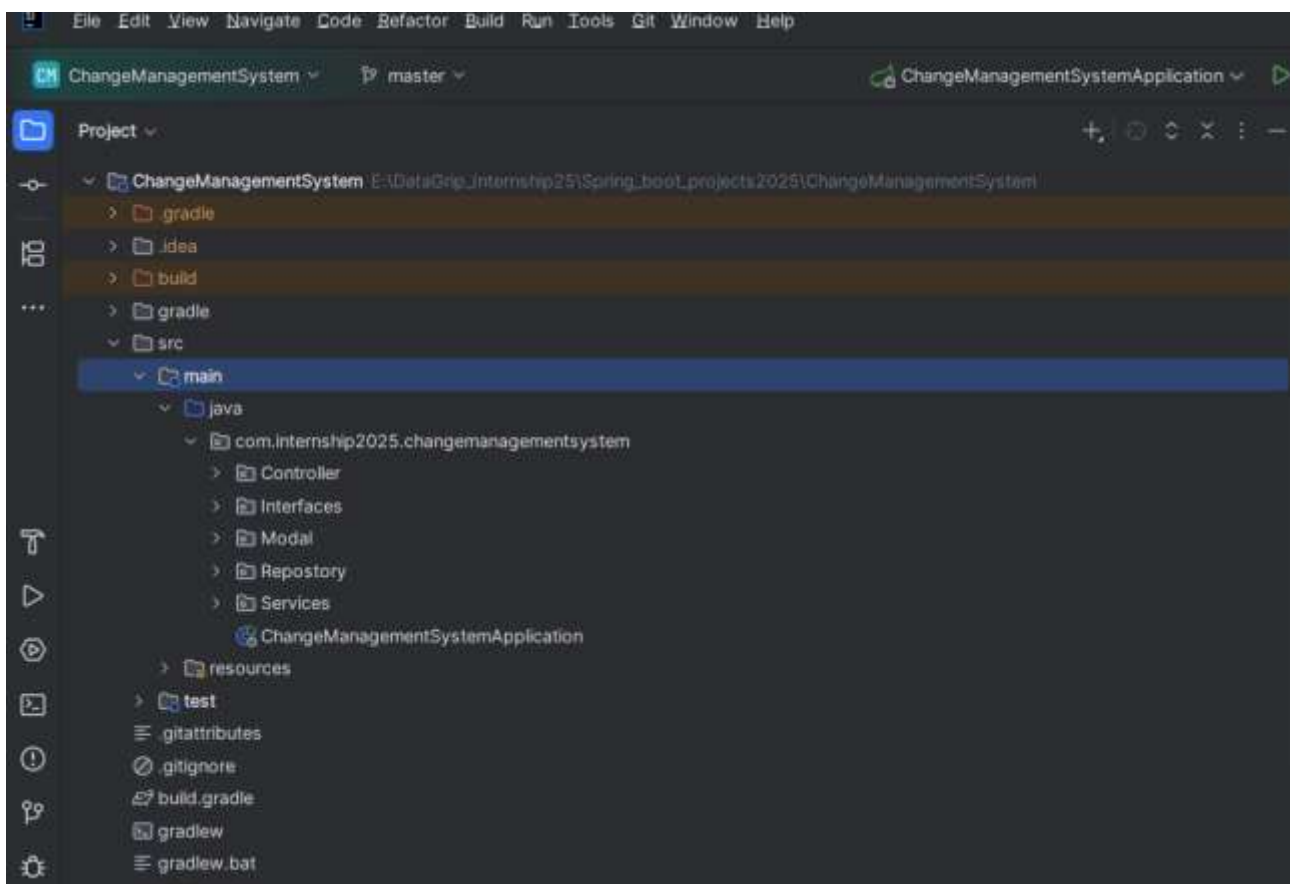


Figure 12

THE MODAL FOLDER

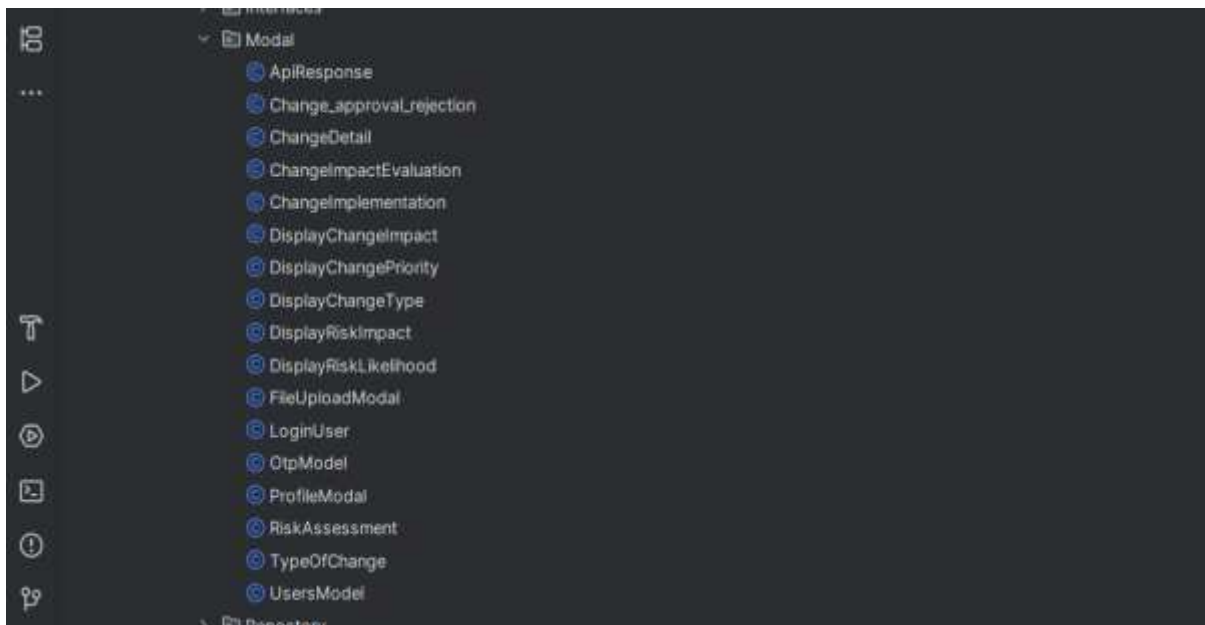


Figure 10

The Controller and Interface Folder



Figure 13

REPOSITORY FOLDER

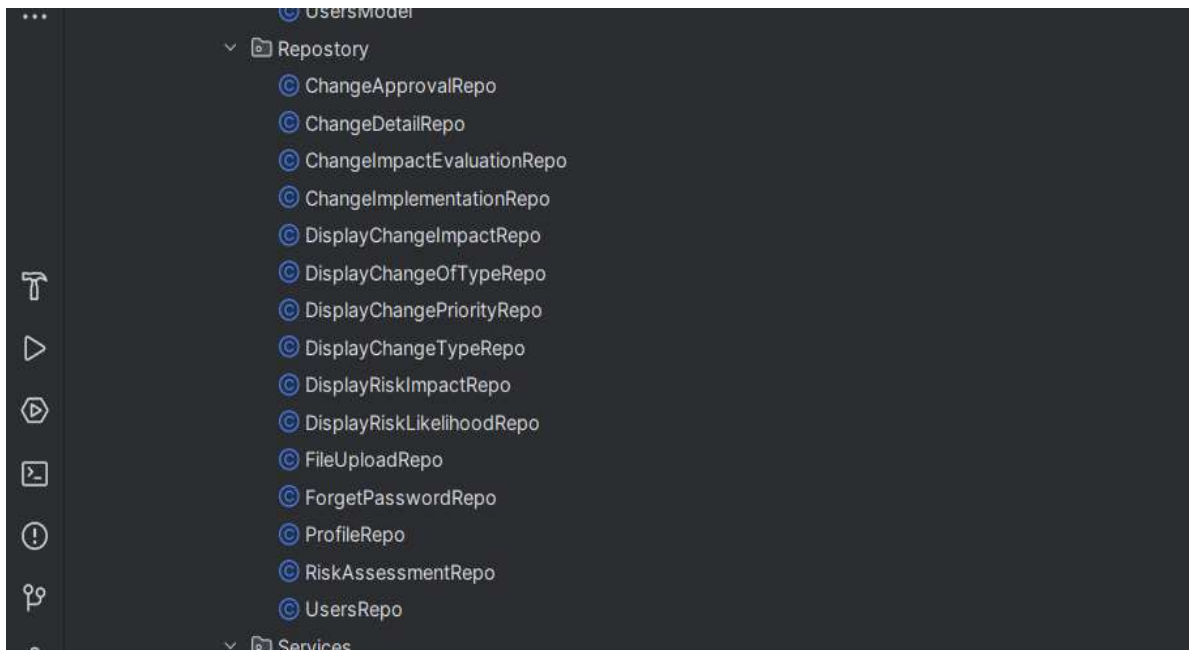


Figure 14

THE SERVICE FOLDER

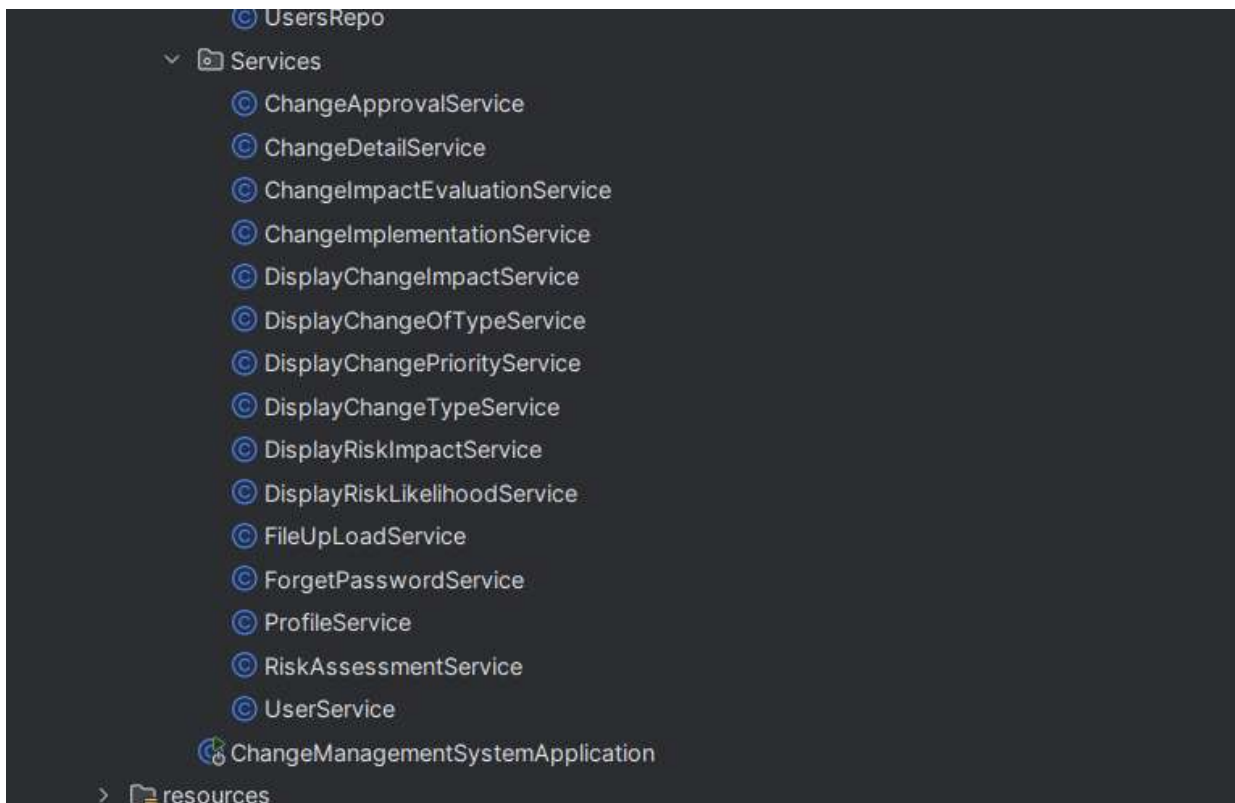


Figure 15

CHANGE MANAGEMENT DATABASE

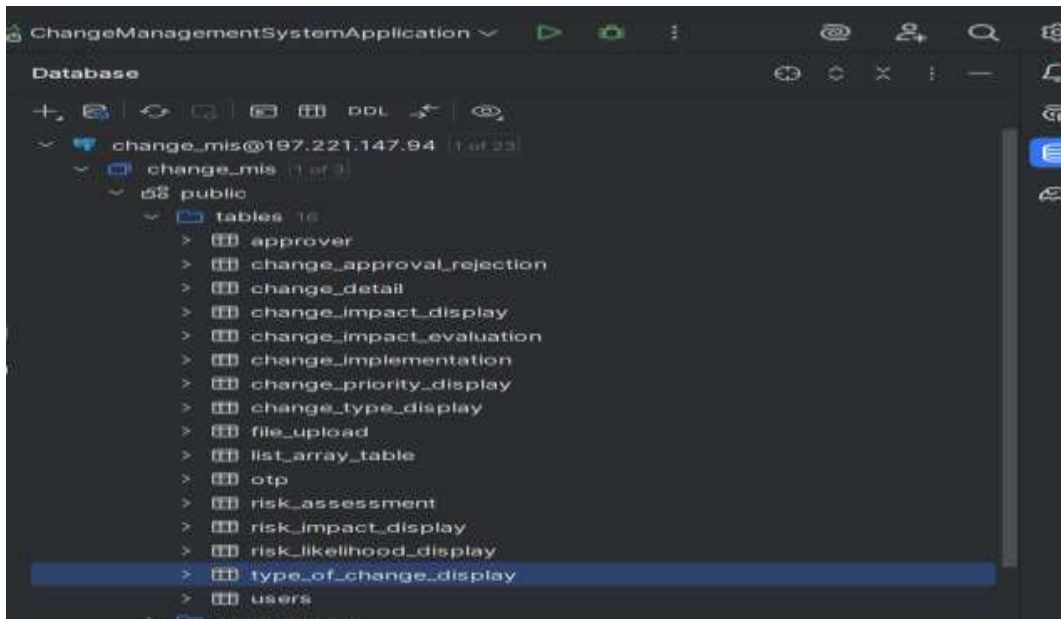


Figure 16

The Project and Database Structure

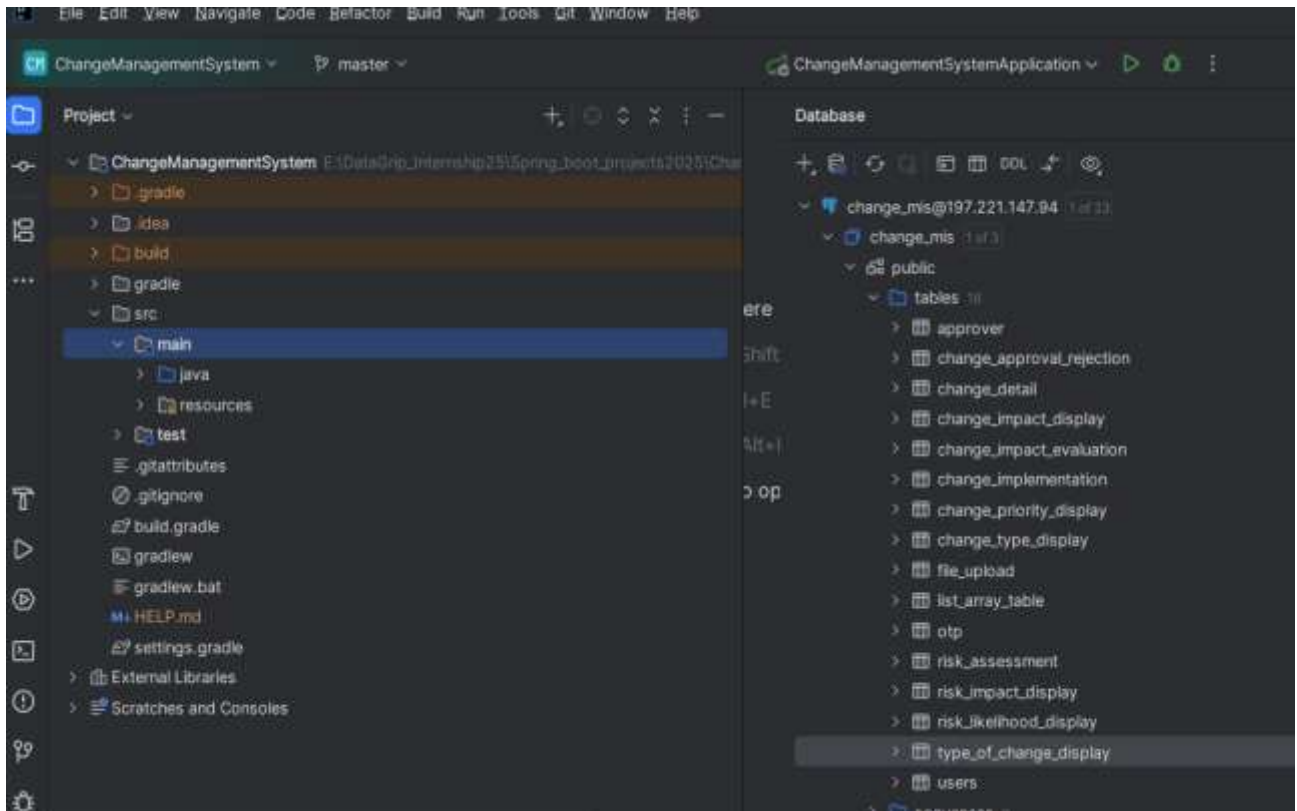


Figure 17

Postman Structure

It is a tool for testing the software of an API. It can be used to design, document, verify, create, and change APIs.

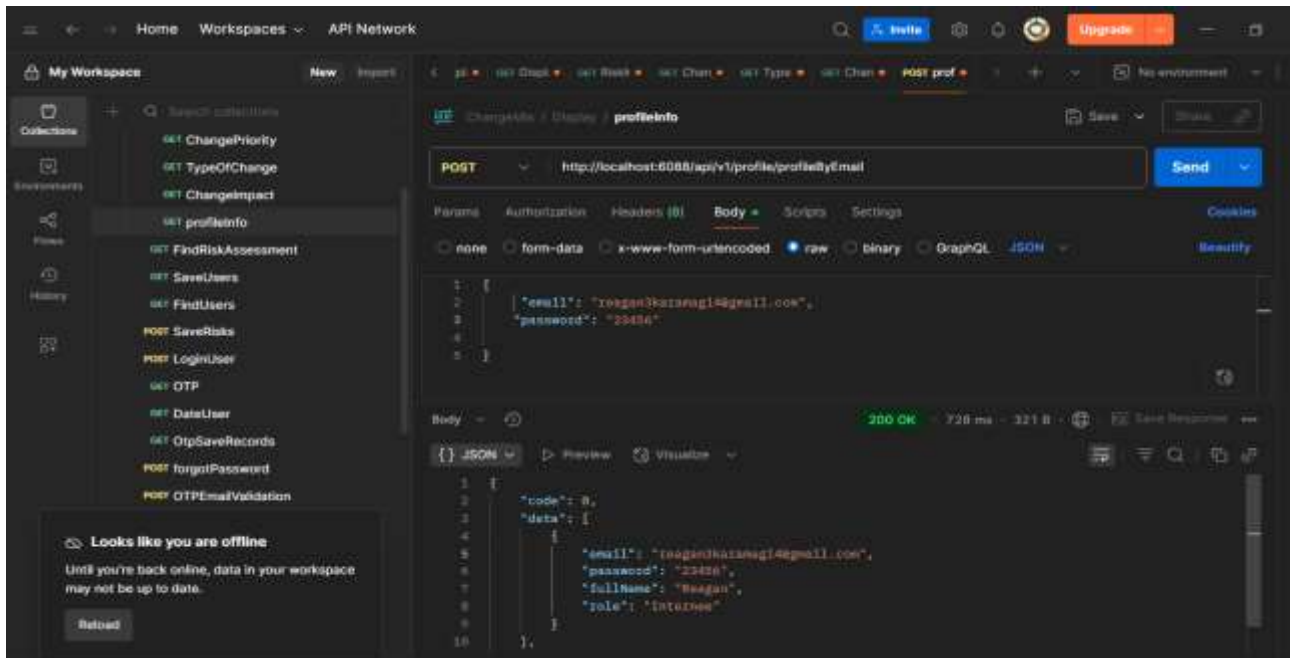


Figure 18

The Project Main Class

A class with a main () method to bootstrap the application

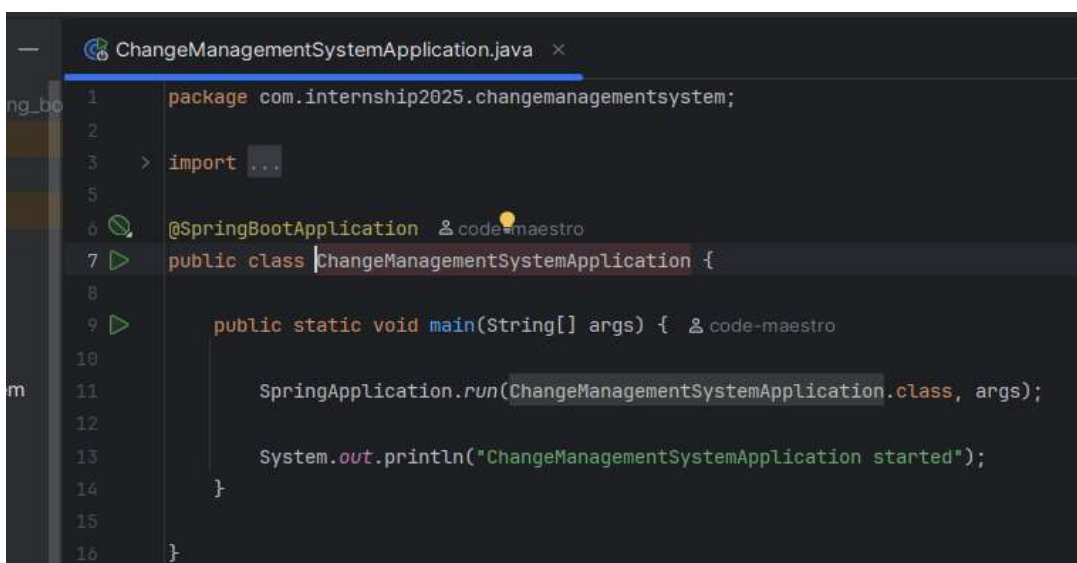
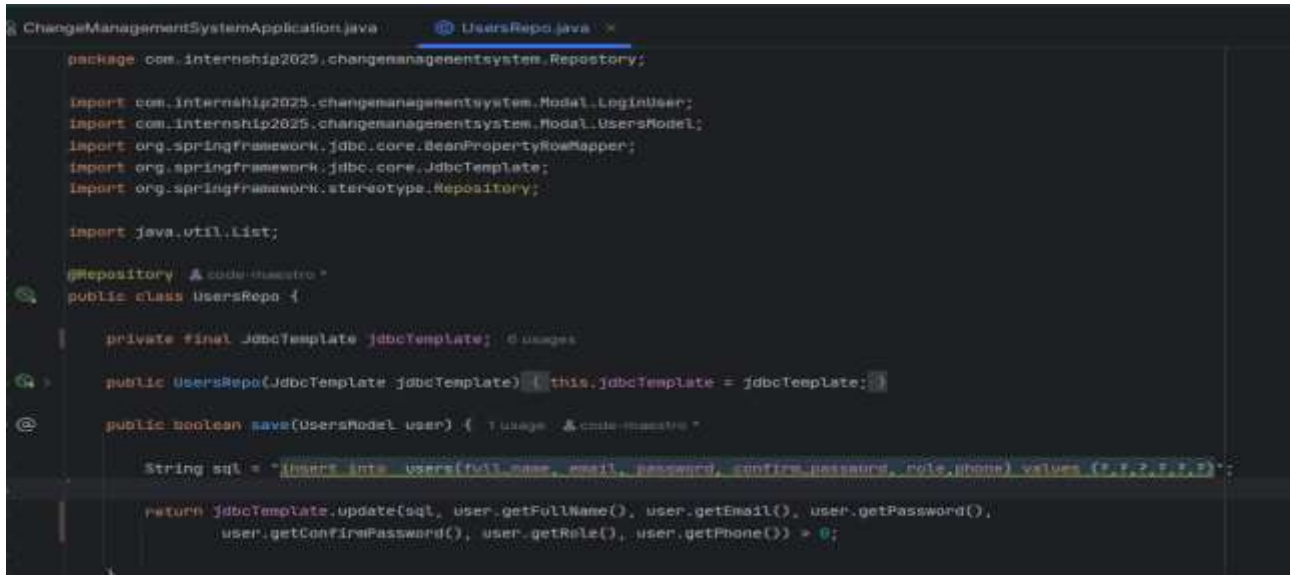


Figure 19

The User Repository Source code

This performs the CRUD (Create, Read, Update and Delete) function on user information.



```
package com.internship2025.changemanagementsystem.Repository;

import com.internship2025.changemanagementsystem.Model.LoginUser;
import com.internship2025.changemanagementsystem.Model.UsersModel;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public class UsersRepo {

    private final JdbcTemplate jdbcTemplate;

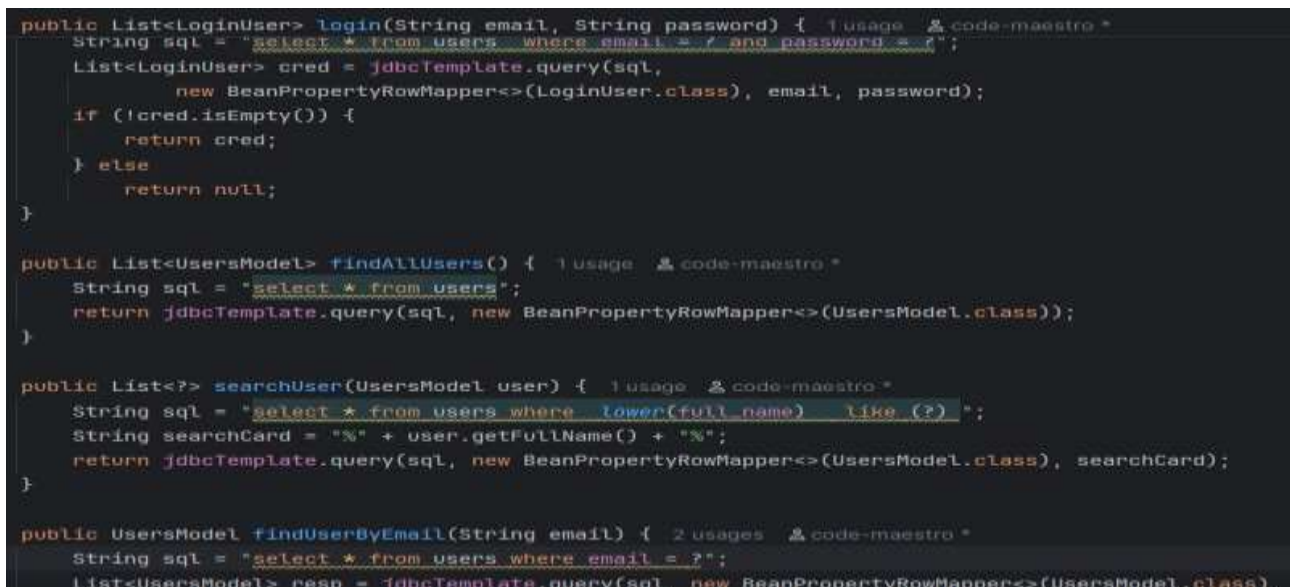
    public UsersRepo(JdbcTemplate jdbcTemplate) { this.jdbcTemplate = jdbcTemplate; }

    public boolean save(UsersModel user) {

        String sql = "insert into users(full_name, email, password, confirm_password, role, phone) values (?, ?, ?, ?, ?, ?)";

        return jdbcTemplate.update(sql, user.getFullName(), user.getEmail(), user.getPassword(),
            user.getConfirmPassword(), user.getRole(), user.getPhone()) > 0;
    }
}
```

Figure 18



```
public List<LoginUser> login(String email, String password) {
    String sql = "select * from users where email = ? and password = ?";
    List<LoginUser> cred = jdbcTemplate.query(sql,
        new BeanPropertyRowMapper<>(LoginUser.class), email, password);
    if (cred.isEmpty()) {
        return cred;
    } else {
        return null;
    }
}

public List<UsersModel> findAllUsers() {
    String sql = "select * from users";
    return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class));
}

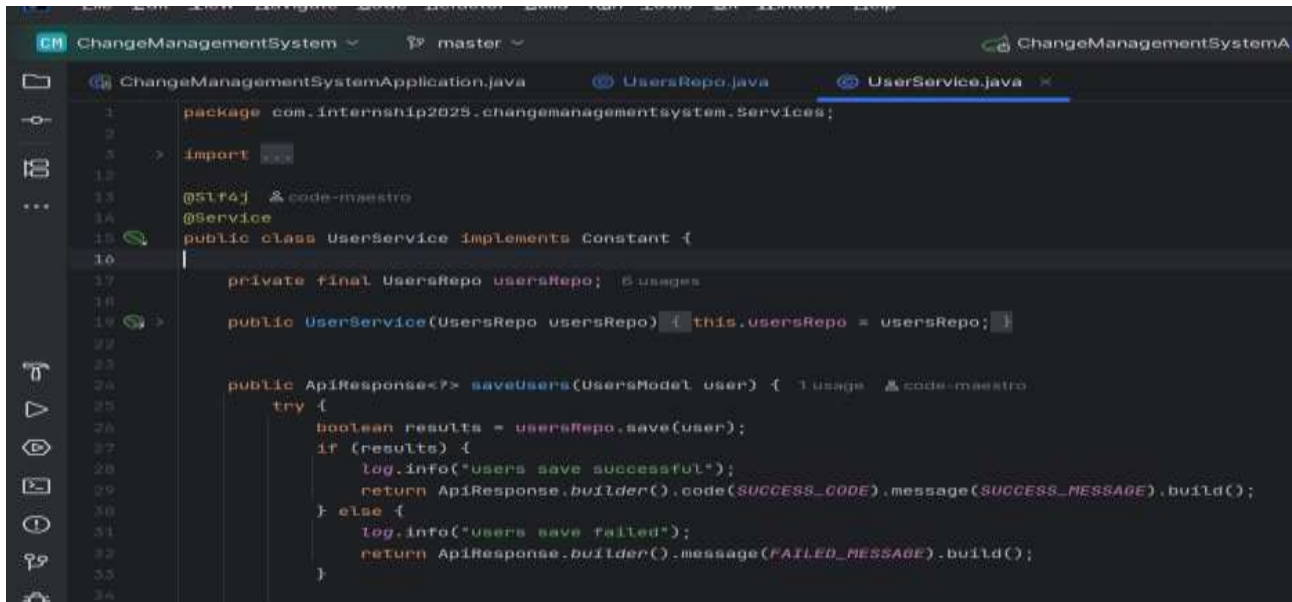
public List<?> searchUser(UsersModel user) {
    String sql = "select * from users where lower(full_name) like (?)";
    String searchCard = "%" + user.getFullName() + "%";
    return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class), searchCard);
}

public UsersModel findUserByEmail(String email) {
    String sql = "select * from users where email = ?";
    List<UsersModel> resp = jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class));
}
```

Figure 20

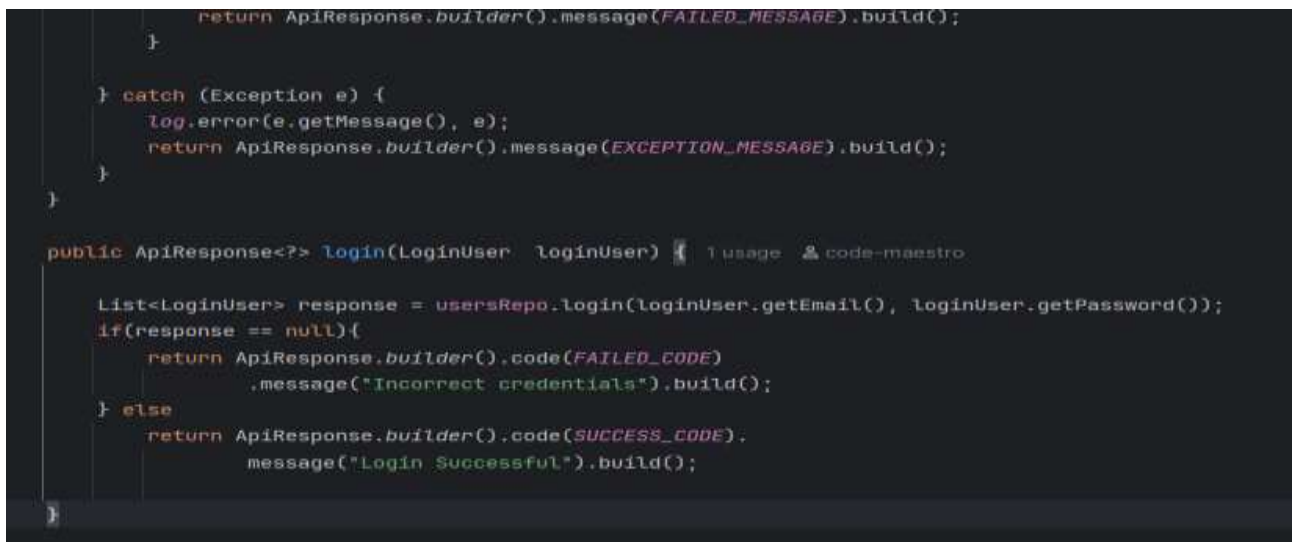
User Service Class

This is a class for business logic on user records. That is, registering, login functions, and lookup to the repository class for database sourcing.



```
1 package com.internship2025.changemanagementsystem.Services;
2
3 import java.util.List;
4
5 @Slf4j
6 @Service
7 public class UserService implements Constant {
8
9     private final UsersRepo usersRepo;
10
11     public UserService(UsersRepo usersRepo) { this.usersRepo = usersRepo; }
12
13     public ApiResponse<?> saveUsers(UsersModel user) {
14         try {
15             boolean results = usersRepo.save(user);
16             if (results) {
17                 log.info("Users save successful");
18                 return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE).build();
19             } else {
20                 log.info("Users save failed");
21                 return ApiResponse.builder().message(FAILED_MESSAGE).build();
22             }
23         } catch (Exception e) {
24             log.error(e.getMessage(), e);
25             return ApiResponse.builder().message(EXCEPTION_MESSAGE).build();
26         }
27     }
28
29     public ApiResponse<?> login(LoginUser loginUser) {
30         List<LoginUser> response = usersRepo.login(loginUser.getEmail(), loginUser.getPassword());
31         if (response == null) {
32             return ApiResponse.builder().code(FAILED_CODE).message("Incorrect credentials").build();
33         } else {
34             return ApiResponse.builder().code(SUCCESS_CODE).message("Login Successful").build();
35         }
36     }
37 }
```

Figure 21



```
return ApiResponse.builder().message(FAILED_MESSAGE).build();
}

} catch (Exception e) {
    log.error(e.getMessage(), e);
    return ApiResponse.builder().message(EXCEPTION_MESSAGE).build();
}

}

public ApiResponse<?> login(LoginUser loginUser) {
    List<LoginUser> response = usersRepo.login(loginUser.getEmail(), loginUser.getPassword());
    if (response == null) {
        return ApiResponse.builder().code(FAILED_CODE).message("Incorrect credentials").build();
    } else {
        return ApiResponse.builder().code(SUCCESS_CODE).message("Login Successful").build();
    }
}
```

Figure 22

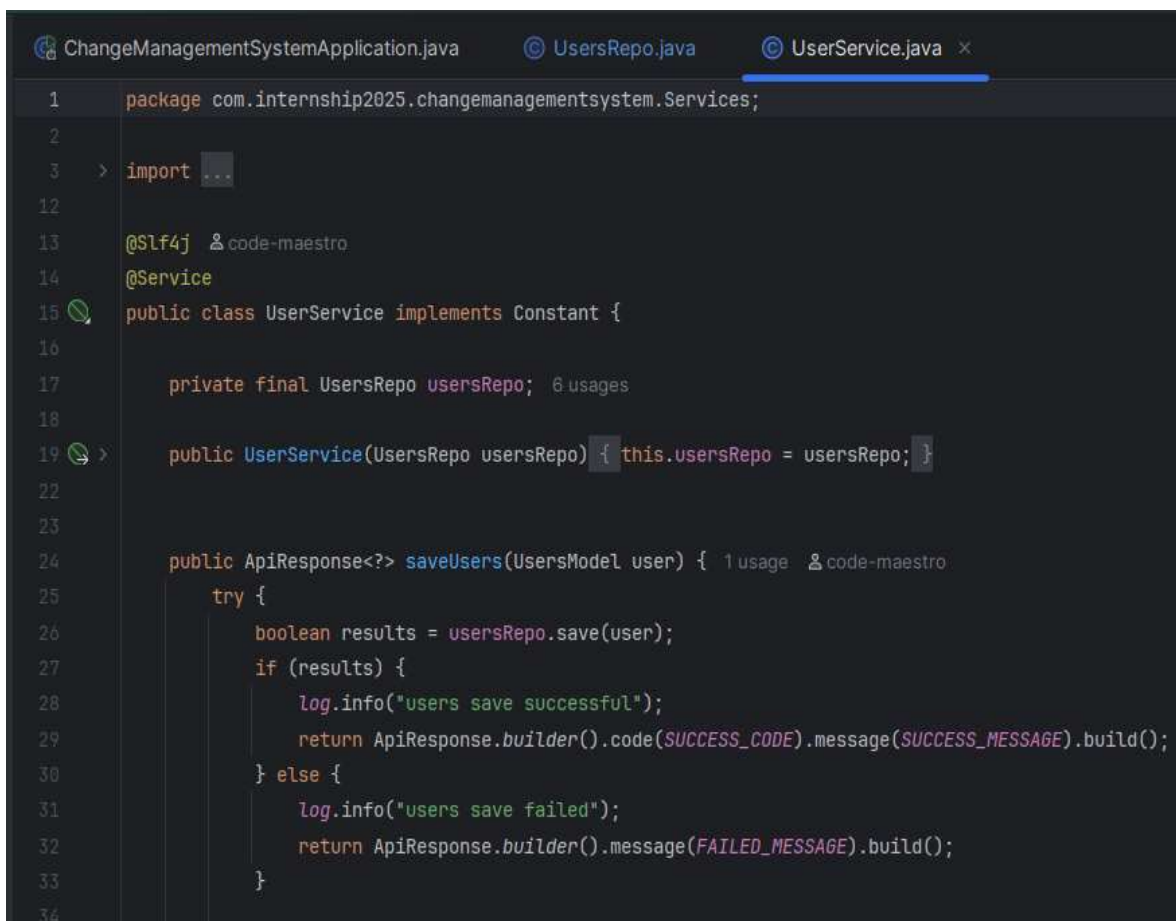
User Service Class Continues

```
public ApiResponse<?> validateLogin(UserModel user) { 1 usage  & code-maestro
    try {
        UserModel responseData = usersRepo.findUserByEmail(user.getEmail());
        if (responseData == null) {
            return ApiResponse.builder().code(NO_DATA_FOUND_CODE)
                .message("Invalid Email").build();
        } else {
            return ApiResponse.builder().code(SUCCESS_CODE)
                .message(SUCCESS_MESSAGE).build();
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return ApiResponse.builder().code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE).build();
    }
}

public ApiResponse<?> findUsers() { 1 usage  & code-maestro
    try {
        List<UserModel> userModelList = usersRepo.findAllUsers();

        if (userModelList.isEmpty()) {
            System.out.println("Users find successful");
        }
    }
}
```

Figure 23



```
package com.internship2025.changemanagementsystem.Services;

import ...

@Slf4j & code-maestro
@Service
public class UserService implements Constant {

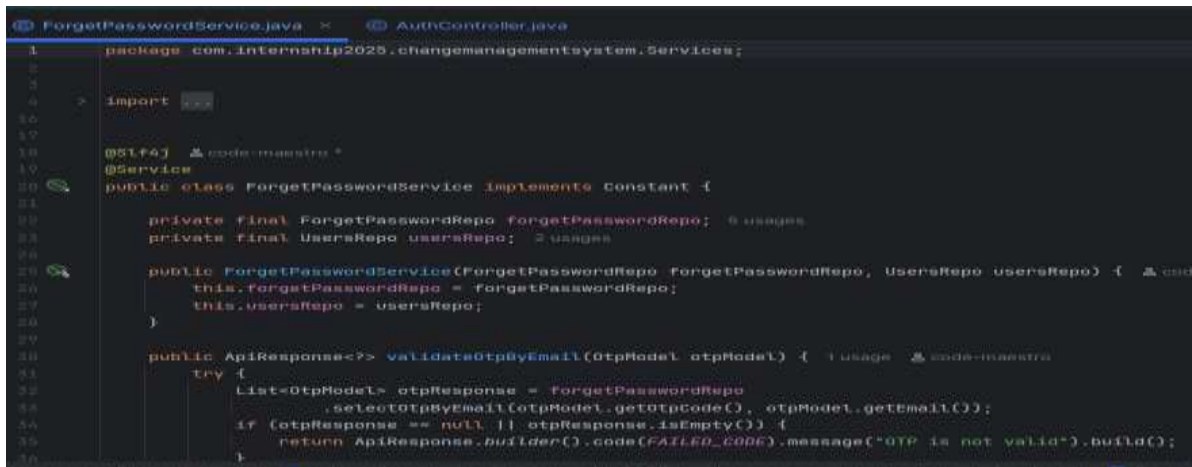
    private final UsersRepo usersRepo; 6 usages

    public UserService(UsersRepo usersRepo) { this.usersRepo = usersRepo; }

    public ApiResponse<?> saveUsers(UserModel user) { 1 usage  & code-maestro
        try {
            boolean results = usersRepo.save(user);
            if (results) {
                log.info("users save successful");
                return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE).build();
            } else {
                log.info("users save failed");
                return ApiResponse.builder().message(FAILED_MESSAGE).build();
            }
        }
    }
}
```

Figure 24

Forget Password Service class



```
1 package com.internship2025.changemanagementsystem.Services;
2
3
4 > Import
5
6
7
8
9
10 @Slf4j
11 @Service
12 public class ForgetPasswordService implements Constant {
13
14     private final ForgetPasswordRepo forgetPasswordRepo;
15     private final UsersRepo usersRepo;
16
17     public ForgetPasswordService(ForgetPasswordRepo forgetPasswordRepo, UsersRepo usersRepo) {
18         this.forgetPasswordRepo = forgetPasswordRepo;
19         this.usersRepo = usersRepo;
20     }
21
22     public ApiResponse<?> validateOtpByEmail(OtpModel otpModel) {
23         try {
24             List<OtpModel> otpResponse = forgetPasswordRepo
25                 .selectOtpByEmail(otpModel.getEmail(), otpModel.getOtpCode());
26             if (otpResponse == null || otpResponse.isEmpty()) {
27                 return ApiResponse.builder().code(FAILED_CODE).message("OTP is not valid").build();
28             }
29         }
30     }
31 }
```

Figure 25



```
    public ApiResponse<?> validateOtpByEmail(OtpModel otpModel) {
        try {
            OtpModel foundOtp = otpResponse.get(0);
            if ("Expired".equalsIgnoreCase(foundOtp.getStatus())) {
                return ApiResponse.builder().code(FAILED_CODE).message("OTP is expired").build();
            }
            boolean statusUpdated = forgetPasswordRepo.updateOtpStatus(otpModel);
            if (!statusUpdated) {
                return ApiResponse.builder().code(FAILED_CODE).message("Failed to update OTP status").build();
            } else {
                return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE).build();
            }
        } catch (Exception e) {
            log.error("OTP validation failed", e);
            return ApiResponse.builder()
                .code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE + ": " + e.getMessage()).build();
        }
    }
}
```

Figure 26

Forget Password Service Continue

The method shown concerns with the logic flow when a user forgets the password.

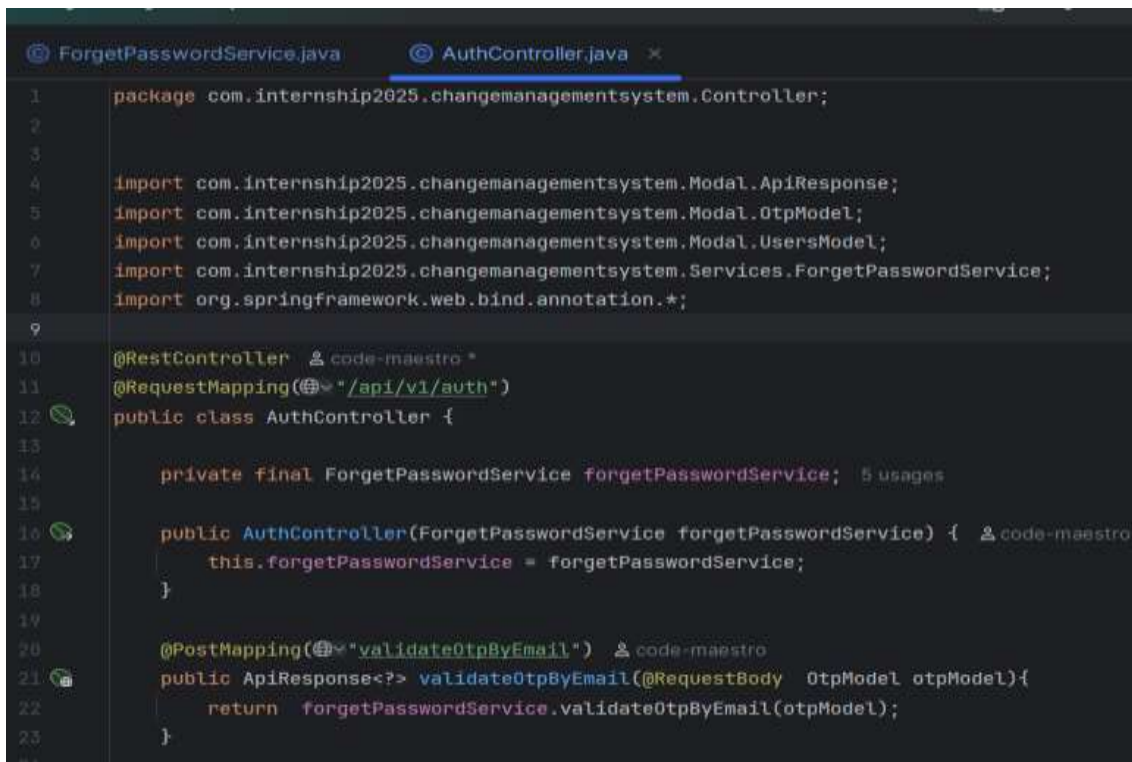
```
public class ForgetPasswordService implements Constant {  
  
    public ApiResponse<?> forgetPassword(String email) { 1 usage: 3 code-maestro *  
        try {  
            if (email == null || email.trim().isEmpty()) {  
                return ApiResponse.builder().code(FAILED_CODE)  
                    .message("email is incorrect").build();  
            }  
            UsersModel usersModel = usersRepo.findUserByEmail(email.trim());  
            if (usersModel == null) {  
                return ApiResponse.builder().code(FAILED_CODE)  
                    .message("Email doesn't exist").build();  
            } else {  
                Integer generatedOTP = OTPNumber();  
                OtpModel otpModel = new OtpModel();  
                otpModel.setEmail(email);  
                otpModel.setOtpCode(generatedOTP);  
                otpModel.setStatus("Pending");  
                otpModel.setCreatedDate(LocalDateTime.now());  
                return saveOtpRecord(otpModel);  
            }  
        } catch (Exception e) {  
            return ApiResponse.builder().code(EXCEPTION_CODE)  
                .message(e.getMessage()).build();  
        }  
    }  
}
```

Figure 27

```
28 public class ForgetPasswordService implements Constant {  
56     public ApiResponse<?> forgetPassword(String email) { 1 usage: 3 code-maestro *  
79         .message(e.getMessage()).build();  
88     }  
81 }  
82  
83 public Integer OTPNumber() { 1 usage: 3 code-maestro  
84     return 100000 + (int) (Math.random() * 900000); // Generates a number between 100000 and 999999  
85 }  
86  
87  
88 public ApiResponse<?> updatePassword(UsersModel usersModel) { 1 usage: 3 code-maestro  
89     try {  
90         boolean updateResponse = forgetPasswordRepo.updatePassword(usersModel);  
91         if (updateResponse) {  
92             return ApiResponse.builder().code(SUCCESS_CODE).message("New password saved").build();  
93         } else {  
94             return ApiResponse.builder().code(FAILED_CODE).message(FAILED_MESSAGE).build();  
95         }  
96     } catch (Exception e) {  
97         return ApiResponse.builder().code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE + e.getMessage()).build();  
98     }  
99 }  
100 }
```

Figure 28

forget controller



```
1 package com.internship2025.changemanagementsystem.Controller;
2
3
4 import com.internship2025.changemanagementsystem.Model.ApiResponse;
5 import com.internship2025.changemanagementsystem.Model.OtpModel;
6 import com.internship2025.changemanagementsystem.Model.UsersModel;
7 import com.internship2025.changemanagementsystem.Services.ForgetPasswordService;
8 import org.springframework.web.bind.annotation.*;
9
10 @RestController @code-maestro *
11 @RequestMapping("/api/v1/auth")
12 public class AuthController {
13
14     private final ForgetPasswordService forgetPasswordService; 5 usages
15
16     public AuthController(ForgetPasswordService forgetPasswordService) { @code-maestro
17         this.forgetPasswordService = forgetPasswordService;
18     }
19
20     @PostMapping("/validateOtpByEmail") @code-maestro
21     public ApiResponse<?> validateOtpByEmail(@RequestBody OtpModel otpModel){
22         return forgetPasswordService.validateOtpByEmail(otpModel);
23     }
24 }
```

Figure 29



```
@PostMapping("/updatePassword") @code-maestro
public ApiResponse<?> updatePassword(@RequestBody UsersModel usersModel){
    return forgetPasswordService.updatePassword(usersModel);
}

@PostMapping("/saveOtpRecord") @code-maestro
public ApiResponse<?> saveOtpRecord(@RequestBody OtpModel otpModel) {
    return forgetPasswordService.saveOtpRecord(otpModel);
}

@PostMapping("/forgetPassword") @code-maestro
public ApiResponse<?> forgetPassword(@RequestBody OtpModel otpModel){
    return forgetPasswordService.forgetPassword(otpModel.getEmail());
}
}
```

Figure 30